TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Campus Tecnológico Central Cartago

Escuela de Ingeniería en Computación

IC-6600 Principios de Sistemas Operativos

Tarea #3

Mr. Meeseeks

Procesos con fork, comunicación por pipes, memoria compartida y semáforos

Oscar Andrés Bezara Pérez

Carné 2024075088

Prof. Esteban Arias-Méndez

Periodo 2-2026

Fecha de entrega: 15 de octubre de 2026

Repositorio: https://github.com/Osi200/SO-T3-Meeseeks

Abstract

This report documents the design and implementation of a Mr. Meeseeks Box in C on Linux. The program receives requests from the user and creates a Meeseeks process with fork for each one. Text requests are simulated with a tree of Meeseeks whose size depends on the task difficulty, arithmetic requests are solved by a single Meeseeks, and external programs are run and judged by their exit status. Processes communicate through pipes, share counters in shared memory protected by a POSIX semaphore, and are terminated in cascade with signals. A process cap and a global time limit prevent impossible tasks from collapsing the system. Experiments compare difficulty, working time and request type, and show that the number of Meeseeks grows sharply as difficulty decreases.

TEC | Tecnológico de Costa Rica

Índice

1. Introducción **1**
2. Descripción general del programa **1**
3. Diseño **2**
3.1 Organización del código 2
3.2 Árbol de procesos y comunicación 2
3.3 Dificultad y fórmula de resolución 4
3.4 Memoria compartida y semáforo 6
3.5 Terminación en cascada con señales 7
3.6 Plan de contingencia: tope de procesos y explosión de meeseeks 8
3.7 Medición del tiempo real 10
4. Implementación de los casos **10**
4.1 Ciclo principal de Box 10
4.2 Caso A: solicitudes de texto 11
4.3 Caso B: cálculos 12
4.4 Caso C: programas externos 13
5. Ejecuciones **14**
5.1 Caso A: solicitudes de texto 14
5.2 Caso B: cálculos 17
5.3 Caso C: programas externos 18
5.4 Tarea imposible y explosión de meeseeks 19
5.5 Informe final de Box 19
6. Experimentos y análisis (0x02) **22**
6.1 Metodología 22
6.2 Experimento 1: dificultad contra cantidad de Meeseeks y tiempo 23
6.3 Experimento 2: tiempo de trabajo fijo contra aleatorio 23
6.4 Experimento 3: costo de cada tipo de solicitud 24
6.5 Experimento 4: tarea imposible y comportamiento del sistema 25
7. Observaciones **26**
8. Conclusiones **26**
9. Uso de Git **27**
Referencias **27**
Apéndice A: Código fuente de meeseeksBoxFinal.c **28**

TEC | Tecnológico de Costa Rica

1. Introducción

Esta tarea consiste en implementar, en lenguaje C sobre Linux, una "Box de Mr. Meeseeks": un programa que recibe solicitudes del usuario y crea procesos (los Meeseeks) para atenderlas. La idea viene del episodio "Meeseeks and Destroy" de Rick and Morty, donde cada Meeseeks existe solo para cumplir una tarea y, si no puede, invoca a más Meeseeks para que lo ayuden, lo que en tareas difíciles termina en un caos de cientos de ellos.

Detrás del tema hay conceptos centrales del curso. Cada Meeseeks es un proceso creado con la llamada al sistema `fork`, los procesos se comunican mediante tuberías ordinarias (`pipe`), los contadores globales viven en memoria compartida protegida por un semáforo, y la terminación coordinada del árbol se resuelve con señales. Además, la tarea pide medir tiempos de ejecución reales y analizar experimentalmente cómo se comporta el sistema según la dificultad de las solicitudes.

El programa soporta los tres tipos de solicitud del enunciado: solicitudes de texto que se simulan con un árbol de Meeseeks cuyo tamaño depende de la dificultad (caso A), cálculos aritméticos y lógicos que resuelve un solo Meeseeks (caso B), y la ejecución de programas externos cuyo valor de retorno determina si la tarea se completó (caso C). También incluye un plan de contingencia para las tareas imposibles, basado en un tope de procesos vivos y en un tiempo máximo de "explosión de meeseeks".

Este documento describe primero el diseño y las decisiones tomadas, luego explica la implementación con los fragmentos de código relevantes, muestra corridas de cada caso y, finalmente, presenta los experimentos de la sección 0x02 con su análisis y conclusiones. Todo el desarrollo se realizó con Git, en el repositorio indicado en la portada.

2. Descripción general del programa

El programa es un único ejecutable, `box`, compilado desde `src/meeseeksBoxFinal.c`. Todos los procesos Meeseeks ejecutan el mismo código: el proceso Box hace `fork` y el hijo resultante entra a la función del Meeseeks, que a su vez puede volver a hacer `fork` para crear ayudantes. Se compila y ejecuta así:

```
gcc -Wall -Wextra -pthread -o box src/meeseeksBoxFinal.c
./box [tope_procesos] [segundos_caos] [-v]
```

El primer argumento es el máximo de Meeseeks vivos al mismo tiempo (por defecto 60), el segundo es el tiempo máximo antes de declarar la explosión de meeseeks (por defecto 300

segundos, es decir, los 5 minutos que pide el enunciado como mínimo) y la opción `-v` muestra la dificultad de cada Meeseeks en su saludo, lo cual solo se usó para depurar.

Una vez dentro, Box muestra el indicador `Box>` y acepta solicitudes una tras otra hasta que el usuario escribe `adios`. El tipo de solicitud se reconoce por su prefijo: las que empiezan con `calc` son cálculos, las que empiezan con `exec` son programas externos y cualquier otro texto es una solicitud simulada. Se eligieron prefijos en lugar de intentar adivinar el tipo porque adivinar es frágil; por ejemplo, "dos cafés" contiene un número y no es un cálculo.

Además hay comandos de configuración que no crean Meeseeks. Con `dif aleatoria`, `dif preguntar` o `dif <0-100>` se elige si la dificultad de las solicitudes de texto se sortea, se le pregunta al usuario en cada solicitud o se fija en un valor, que cubre los dos casos que define el enunciado y además permite repetir experimentos con la misma dificultad. Con `tiempo aleatorio` o `tiempo <segundos>` se elige si el tiempo de trabajo de cada Meeseeks se sortea entre 0.5 y 5 segundos o se fija en un valor definido por el usuario.

Al despedirse, Box imprime un informe final con todas las solicitudes atendidas: su texto, si se completó, si terminó por explosión de meeseeks, la duración total, la duración real sin las esperas simuladas y la cantidad de Meeseeks que se crearon para resolverla.

3. Diseño

3.1 Organización del código

El archivo final está dividido en once secciones marcadas con comentarios: inclusiones y constantes; tipos de datos y variables globales; utilidades generales (reloj y números aleatorios); memoria compartida y semáforo; el evaluador de operaciones del caso B; la dificultad, el tiempo y la fórmula del árbol; la terminación en cascada con señales; el árbol de Meeseeks con pipes; los tres tipos de solicitud; las funciones de apoyo de Box; y el programa principal. Las versiones intermedias del desarrollo se conservan en `src/versiones/` para mostrar la evolución del programa, y el simulador usado para ajustar la fórmula está en `src/sim.c`.

3.2 Árbol de procesos y comunicación

La estructura del caso A es un árbol. Box crea con `fork` al Meeseeks raíz (nivel N = 1, instancia i = 1) y lo espera. Si el raíz no puede resolver la tarea, crea 5 o 2 hijos en el nivel N + 1, numerados con i = 1, 2, 3, ..., y cada hijo repite exactamente la misma lógica. El árbol se forma por recursión: la función del Meeseeks se llama a sí misma dentro de cada hijo, y un Meeseeks nunca vuelve al ciclo de Box, porque al terminar su trabajo hace `exit`.

Entre cada padre y cada hijo hay dos tuberías ordinarias. Como un pipe es unidireccional, hace falta una para cada sentido. Por el **pipe de ida** el padre le envía al hijo la solicitud del usuario, como mensaje privado, tal como exige el enunciado. Por el **pipe de vuelta** el hijo le devuelve al padre un reporte con el resultado de su subárbol: si alguien resolvió la tarea y, en ese caso, el pid, ppid, nivel e instancia del Meeseeks que la resolvió. De esta forma la información sube nivel por nivel hasta el raíz sin usar variables globales para transmitir mensajes.

Cuando un Meeseeks resuelve la tarea, su reporte llega al padre, el padre les envía la señal SIGTERM a los hermanos que siguen vivos para que se despidan, y luego el padre se despide y reenvía el reporte a su propio padre. La despedida se propaga recursivamente hacia atrás hasta el Meeseeks original, que informa al usuario quién resolvió la solicitud. Los contadores globales viven en una región de memoria compartida protegida por un semáforo. La figura 1 resume todo este diseño.

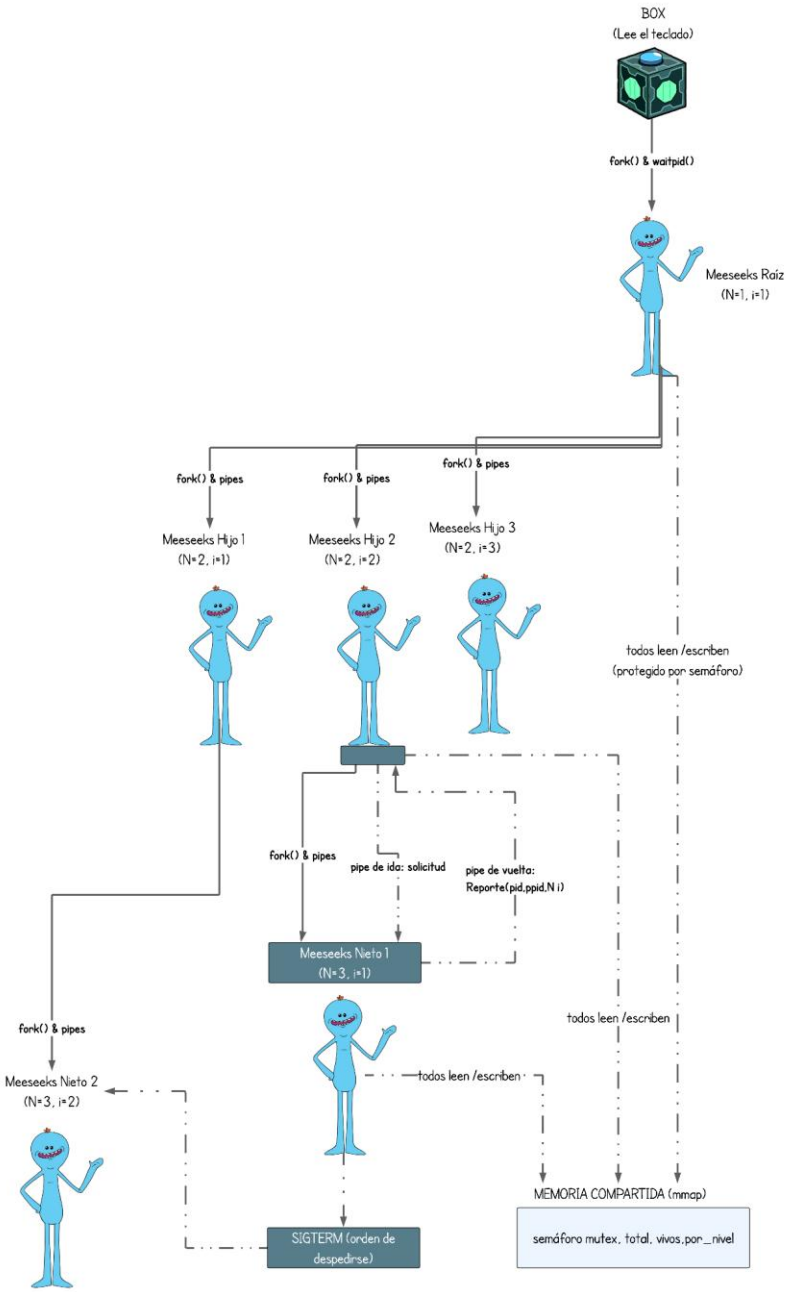


Figura 1. Árbol de Meeseeks: creación con fork, comunicación por pipes y terminación en cascada.

3.3 Dificultad y fórmula de resolución

La dificultad D es un número entre 0 y 100 que se interpreta como la probabilidad de que la tarea pueda resolverse: 100 es una tarea muy fácil y 0 es una tarea imposible. Cuando la dificultad se deja a la suerte se sortea con una distribución uniforme entre 0 y 100. Se consideró usar una

distribución normal centrada en 50, pero no hay ningún dato que justifique sus parámetros, y la uniforme tiene la ventaja de producir con la misma frecuencia tareas de todos los rangos, lo cual facilita observar el comportamiento del sistema en cada zona.

La fórmula que aplica cada Meeseeks, después de su tiempo de trabajo simulado, es la siguiente. Si D es mayor que 85, resuelve la tarea él solo, como establece el enunciado. Si no, "intenta" resolverla tirando un dado: genera un número aleatorio entre 0 y 100 y gana si ese número es menor que D multiplicado por un factor de atenuación llamado `NERF_DADO`, que vale 0.1. Si el dado falla, pide ayuda: crea 5 hijos si D es menor o igual a 45 y 2 hijos si D está entre 45 y 85.

A cada hijo le pasa una dificultad aumentada mediante aging, D + `AGING`, con `AGING` igual a 5 y recortada a un máximo de 100. Es la misma idea del aging en planificación: la tarea se diluye entre los Meeseeks y se vuelve un poco más fácil en cada nivel, hasta que D supera 85 y algún Meeseeks la resuelve con seguridad. El caso imposible sale solo de la fórmula, sin programar un caso especial: si D vale 0, el aging no la modifica (0 se queda en 0) y el dado nunca acierta, porque ningún número es menor que 0. Por eso una tarea de dificultad 0 nunca se resuelve y su árbol crecería sin fin si no existiera el plan de contingencia.

```c
/* Aging: dificultad que reciben los hijos */
static double dificultad_hijo(double D) {
    double D_hijo;

    if (D == 0) {
        D_hijo = 0;            /* imposible: nunca se simplifica */
    } else {
        D_hijo = D + AGING;    /* la tarea se simplifica un poco por nivel */
    }

    if (D_hijo > 100.0) {
        D_hijo = 100.0;        /* 100 es el máximo de la escala */
    }
    return D_hijo;
}
```

Los valores de `AGING` y `NERF_DADO` no se eligieron al azar; se ajustaron con un simulador (`src/sim.c`) que recorre el árbol sin crear procesos y cuenta cuántos Meeseeks se generarían en promedio en 1000 pruebas. La tabla 1 resume ese proceso de ajuste.

Configuración probada	D = 5	D = 15	D = 30	Resultado
Aging multiplicativo (D × 2), sin atenuación	4.0	2.7	2.0	Árboles demasiado pequeños
Aging aditivo (+3.14), sin atenuación	6.3	4.3	3.0	Mejora, pero insuficiente

Configuración probada	D = 5	D = 15	D = 30	Resultado
Aging +0.001 y dado × 0.001	~498 (D=2)	483.6	465.3	Casi todo choca con el tope
Aging +1 y dado × 0.1	83.9 (D=2)	70.7	56.1	Curva gradual

Tabla 1. Meeseeks promedio por tarea según el simulador secuencial (tope de 500).

El primer resultado mostró que multiplicar la dificultad por 2 la lleva a 100 en muy pocos niveles, así que incluso las tareas "difíciles" se resolvían con 3 o 4 Meeseeks, lejos de los cientos que sugiere el enunciado. La atenuación del dado fue la que más efecto tuvo: con el dado completo, cinco hermanos con un 15% de probabilidad cada uno casi siempre resuelven en los primeros niveles. Con el dado atenuado, la dificultad se "paga" en profundidad del árbol y no en suerte.

Al pasar de la simulación a procesos reales apareció una diferencia importante. El simulador recorre a los hijos uno por uno y se detiene en el primer éxito, mientras que en el programa real todos los hermanos trabajan en paralelo y siguen creando ayudantes hasta que llega la señal de cancelación. Con `AGING` igual a 1, una corrida de prueba llegó al nivel 16 y tardó 55 segundos, así que el valor final se subió a 5. Con ese ajuste, las tareas resolubles generan entre decenas y unos cientos de Meeseeks y la tarea imposible queda controlada por el tope.

3.4 Memoria compartida y semáforo

Después de un `fork`, cada proceso tiene su propia copia de las variables, así que un contador global normal no sirve: si un hijo le suma 1, el padre no lo ve. Por eso los contadores (Meeseeks creados, nivel máximo, Meeseeks por nivel y Meeseeks vivos) están en una estructura creada con `mmap` y la opción `MAP_SHARED` antes del primer `fork`. Con esa opción, el `fork` no copia la región, sino que todos los procesos apuntan a la misma memoria física.

```c
}

/*---------------------------------------------------------------------------
 * SECCIÓN 4: MEMORIA COMPARTIDA Y SEMÁFORO
 *---------------------------------------------------------------------------*/

/* Se llama UNA vez en Box, antes de cualquier fork: los hijos heredan el mapeo */
static void iniciar_compartido(void) {
    /* MAP_SHARED: el fork NO copia esta región, la comparte con los hijos */
    comp = mmap(NULL, sizeof(Compartido), PROT_READ | PROT_WRITE,
                MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (comp == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }
    /* 2.º argumento = 1: el semáforo se comparte entre procesos.
       3.er argumento = 1: valor inicial 1, es decir, "libre". */
    sem_init(&comp->mutex, 1, 1);
}

/* Contadores en cero antes de cada solicitud nueva */
static void reiniciar_contadores(void) {
    comp->total = 0;
    comp->max_nivel = 0;
    comp->vivos = 1;            /* el Meeseeks raíz */
    comp->espera_final = 0;
    memset(comp->por_nivel, 0, sizeof(comp->por_nivel));
}

/*
```

Como varios Meeseeks modifican esos contadores al mismo tiempo, cada acceso es una sección crítica protegida por un semáforo binario. Se usó un semáforo POSIX sin nombre (`sem_init` con el segundo argumento en 1, es decir, compartido entre procesos) guardado dentro de la propia región compartida. Es la alternativa moderna a los semáforos de System V que presenta el capítulo 14 de Beginning Linux Programming: cumple la misma función de exclusión mutua con operaciones P y V, requiere menos código y no deja semáforos huérfanos en el sistema si el programa termina de forma anormal. Se enlaza con `-pthread`, en línea con la recomendación del enunciado de usar la biblioteca pthreads.

Un detalle importante es que, mientras un proceso tiene el semáforo tomado, la señal SIGTERM queda bloqueada. Si llegara justo en ese momento, el manejador terminaría el proceso con el semáforo cerrado y ningún otro Meeseeks podría volver a entrar a la sección crítica, lo que congelaría todo el árbol. Con la señal bloqueada, el manejador solo se ejecuta después de liberar el semáforo.

El número de nivel N viaja como parámetro de padre a hijo y la instancia i la asigna el padre según el orden de creación de sus hijos (1, 2, 3, ...). Ambos valores son locales a cada familia de hermanos, así que no necesitan memoria compartida. Lo que sí requiere estar compartido y protegido son los contadores globales del árbol.

3.5 Terminación en cascada con señales

Para avisar a los hermanos de que alguien ya resolvió la tarea había dos opciones: enviarles un mensaje por pipe o una señal. Un pipe obligaría a cada Meeseeks a estar revisando su entrada mientras trabaja. Una señal, en cambio, interrumpe al proceso aunque esté dormido en su espera simulada o esperando a sus propios hijos, por lo que se eligió SIGTERM con un manejador propio.

```c

static void al_terminar(int sig) {
    (void)sig;

    /* Propaga la señal hacia abajo */
    for (int k = 0; k < num_hijos; k++) {
        kill(mis_hijos[k], SIGTERM);
    }

    /* Espera a que los hijos se despidan primero */
    while (wait(NULL) > 0) {
        /* nada: solo recogemos hijos */
    }

    /* Si ya me despedí como resolvedor, no repito la despedida */
    if (!despedido) {
        char msg[128];
        int n;

        despedido = 1;
        n = snprintf(msg, sizeof(msg),
                     "[pid %d, ppid %d] Me piden terminar. ¡Gracias por servirte, adiós!\n",
                     getpid(), getppid());
        enviar(STDOUT_FILENO, msg, n);
    }

    desregistrar();
    _exit(EXIT_FAILURE);
}

```

Cuando un Meeseeks recibe SIGTERM, primero reenvía la señal a todos sus hijos y espera con `wait` a que se despidan, y solo después se despide él. Así la despedida ocurre de abajo hacia arriba y ningún hijo queda huérfano. En una versión anterior, sin ese `wait`, se observaron Meeseeks adoptados por otro proceso del sistema porque su padre moría antes que ellos. Dentro del manejador se usan `write` y `_exit` en lugar de `printf` y `exit`, porque solo algunas funciones son seguras dentro de un manejador de señales y `_exit` no vacía los buffers heredados del padre.

Dos detalles evitan errores sutiles. Primero, SIGTERM se bloquea alrededor de cada `fork`, de modo que la señal no puede llegar a un hijo recién nacido que todavía no instaló su manejador ni a un padre que aún no anotó al hijo en su lista. Segundo, la bandera `despedido` evita que un Meeseeks que acaba de resolver y recibe SIGTERM casi al mismo tiempo se despida dos veces.

3.6 Plan de contingencia: tope de procesos y explosión de meeseeks

Una tarea de dificultad 0 nunca se resuelve, así que sin control su árbol crecería hasta agotar los recursos de la máquina. El plan de contingencia tiene dos partes. La primera es un **tope de**

Meeseeks vivos: antes de crear sus hijos, cada Meeseeks pide cupo para todos ellos de una sola vez. La reserva se hace dentro del semáforo y antes del `fork`, para que dos padres no puedan ver al mismo tiempo que queda un lugar libre y ambos lo ocupen. Si no hay cupo, el Meeseeks no puede pedir ayuda y se queda dormido con `pause`, sin consumir procesador, hasta que llegue una señal.

```c
static int reservar_hijos(int n) {
    sigset_t previo;
    int hay_cupo;

    bloquear_term(&previo);
    sem_wait(&comp->mutex);

    if (comp->vivos + n <= comp->max_procs) {
        comp->vivos += n;
        hay_cupo = 1;
    } else {
        hay_cupo = 0;
    }

    sem_post(&comp->mutex);
    sigprocmask(SIG_SETMASK, &previo, NULL);
    return hay_cupo;
}
```

La segunda parte es la **explosión de meeseeks**: Box espera al Meeseeks raíz revisando cada 20 milisegundos si terminó y si se cumplió el tiempo máximo configurado. Si se cumple, Box le envía SIGTERM al raíz y la cascada de la sección anterior termina todo el árbol de forma ordenada. La solicitud se registra como "explosión de meeseeks" en el informe.

```c
        if (r == pid) {
            /* El Meeseeks raíz terminó: éxito si salió normal con código 0 */
            if (WIFEXITED(estado) && WEXITSTATUS(estado) == 0) {
                *ok = 1;
            }
            return;
        }

        if (ahora() > limite) {
            printf("CAOS PLANETARIO: se acabó el tiempo (%d s). Terminando todo.\n", caos_seg);
            fflush(stdout);
            kill(pid, SIGTERM);        /* la cascada baja por todo el árbol */
            waitpid(pid, &estado, 0);  /* espera a que el raíz termine de despedirse */
            *caos = 1;
            return;
        }

        usleep(20000);   /* revisa cada 20 ms sin gastar CPU */
    }
}
```

Como la reserva es "todo o nada" (5 o 2 hijos a la vez), el tope real queda un poco por debajo del configurado: con un tope de 60 el árbol se detuvo en 56 Meeseeks y con un tope de 300 en 296. También queda un árbol desbalanceado, porque el cupo lo obtiene quien lo pida primero, sin importar el nivel.

3.7 Medición del tiempo real

Los tiempos se miden con `clock_gettime(CLOCK_MONOTONIC)`, un reloj que solo avanza hacia adelante y no se ve afectado si cambia la hora del sistema, lo cual es frecuente en una máquina virtual. El enunciado pide no contar las esperas aleatorias de la simulación. Como los Meeseeks trabajan en paralelo, sus esperas se solapan y no se pueden simplemente sumar y restar. Lo que realmente aparece en la duración total es la espera de la cadena de Meeseeks que va del raíz al que resolvió, es decir, la ruta crítica. Por eso cada Meeseeks suma su espera al reporte mientras este sube por el árbol, y Box resta del tiempo total la espera acumulada en esa ruta.

Esta medición tiene dos limitaciones que conviene declarar. En las tareas que no se completan no existe una ruta ganadora, así que su duración "real" incluye las esperas. Además, como Box revisa el estado del Meeseeks raíz cada 20 milisegundos, todas las duraciones tienen una resolución de unos 20 ms, lo cual se discute en los experimentos.

4. Implementación de los casos

4.1 Ciclo principal de Box

Box es un ciclo que lee una línea, le quita el salto de línea, atiende los comandos de configuración y, para las solicitudes, crea al Meeseeks raíz. Antes de cada `fork` se vacía el buffer de salida con `fflush`, porque de lo contrario el hijo heredaría texto pendiente y este aparecería duplicado.

```c
/* Contadores en cero antes de cada solicitud nueva */
static void reiniciar_contadores(void) {
    comp->total = 0;
    comp->max_nivel = 0;
    comp->vivos = 1;            /* el Meeseeks raíz */
    comp->espera_final = 0;
    memset(comp->por_nivel, 0, sizeof(comp->por_nivel));
}
```

El resultado de cada Meeseeks raíz se comunica a Box mediante su código de salida, igual que en los programas de UNIX: `exit(EXIT_SUCCESS)` significa que la tarea se completó y `exit(EXIT_FAILURE)`, que no. Box lo lee con las macros `WIFEXITED` y `WEXITSTATUS`.

4.2 Caso A: solicitudes de texto

El Meeseeks raíz obtiene la dificultad inicial (sorteada, fija o elegida por el usuario), instala su manejador de SIGTERM y llama a la función recursiva del árbol. Cada Meeseeks del árbol se registra en la memoria compartida, saluda con el formato del enunciado, simula su tiempo de trabajo y aplica la fórmula de la sección 3.3.

```c
/* Saludo de los Meeseeks de los casos B y C (el caso A saluda en meeseeks_arbol) */
static void saludar(int N, int i) {
    registrar(N);
    printf("Hi, I'm Mr. Meeseeks! Look at Meeeee. (%d, %d, %d, %d)\n",
           getpid(), getppid(), N, i);
    fflush(stdout);
}
```

La creación de hijos es la parte central del programa. Por cada hijo se crean los dos pipes antes del `fork` para que el hijo los herede. Cada proceso cierra los extremos que no usa, lo cual es indispensable: una lectura solo detecta el fin de archivo cuando todos los extremos de escritura están cerrados, así que un extremo olvidado haría que el padre esperara para siempre. El hijo lee la solicitud del pipe de ida y no de la memoria copiada por el `fork`, y reinicia su generador aleatorio con su propio pid para que los hermanos no tiren el mismo dado.

```
if (pipe(ida[k]) < 0 || pipe(vuelta[k]) < 0) { ... }
sigprocmask(SIG_BLOCK, &bloq, &previo);
p = fork();
```

```
if (p == 0) {                                   /* HIJO */
    close(ida[k][1]);                           /* no escribe en la ida */
    close(vuelta[k][0]);                        /* no lee de la vuelta */
    n = read(ida[k][0], buf, sizeof(buf) - 1); /* recibe la solicitud */
    srand(getpid());
    r = meeseeks_arbol(buf, D_hijo, N + 1, k + 1);
    enviar(vuelta[k][1], &r, sizeof(r));       /* reporta al padre */
    exit(...);
}
/* PADRE */
mis_hijos[num_hijos++] = p;
close(ida[k][0]);
enviar(ida[k][1], solicitud, strlen(solicitud));
close(ida[k][1]);                               /* avisa "fin del mensaje" */
close(vuelta[k][1]);
```

Una vez creados todos los hijos, el padre vigila los pipes de vuelta de todos a la vez con `poll`. Así reacciona al primer hijo que reporte un éxito, sin importar su posición, y en ese momento les envía SIGTERM a los hermanos que todavía no han reportado. Al final recoge a todos los hijos con `wait` para que no queden procesos zombie.

```
n = read(fds[k].fd, &r, sizeof(r));
if (n == sizeof(r) && r.resuelto && !final.resuelto) {
    final = r;                                  /* primer éxito */
    for (int j = 0; j < creados; j++) {
        if (vivo[j]) {
            kill(mis_hijos[j], SIGTERM);        /* cancela a los hermanos */
        }
    }
}
```

4.3 Caso B: cálculos

Para los cálculos se escribió un evaluador por descenso recursivo. Hay una función por cada nivel de precedencia (OR, XOR y AND bit a bit, suma y resta, multiplicación, división y módulo, operadores unarios y, por último, números y paréntesis) y cada una pide sus operandos a la función del nivel inmediatamente más fuerte. Esa cascada es la que hace que `3+4*2` dé 11 y no 14. Un paréntesis reinicia la precedencia evaluando su contenido desde el nivel más bajo, y los

operadores unarios se aplican de forma recursiva, por lo que `--5` vale 5.

```
static long long termino(void) {
    long long v = unario();

    while (!hubo_error) {
        char op;
        long long d;

        saltar();
        op = *cur;
        if (op != '*' && op != '/' && op != '%') {
            break;
        }
        cur++;
        d = unario();

        if (op == '*') {
            v = v * d;
        } else if (d == 0) {
            error("división entre cero");
            return 0;
        } else if (op == '/') {
            v = v / d;
        } else {
            v = v % d;
        }
    }
    return v;
}
```

El evaluador trabaja con enteros de 64 bits y detecta tres tipos de error: división (o módulo) entre cero, expresiones incompletas como `2+` o paréntesis sin cerrar, y caracteres sobrantes como en `2 3`. Un solo Meeseeks atiende el cálculo: si sale bien imprime el resultado y termina con código 0, y si hay un error lo reporta y termina con código 1.

4.4 Caso C: programas externos

La llamada `exec` reemplaza todo el proceso por el programa nuevo, así que si el Meeseeks la usara directamente desaparecería sin poder reportar nada. Por eso el Meeseeks hace su propio `fork`: el hijo ejecuta el programa con `execvp` y el Meeseeks lo espera con `waitpid` para leer su valor de retorno. Quedan tres niveles de procesos: Box, el Meeseeks y el programa externo. La

solicitud se separa por espacios con `strtok` para construir el arreglo de argumentos terminado en `NULL` que exige `execvp`.

```
pid_ext = fork();
if (pid_ext == 0) {
    execvp(args[0], args);   /* si funciona, nunca retorna */
    _exit(127);              /* 127: comando no encontrado */
}
waitpid(pid_ext, &est, 0);
if (WIFEXITED(est) && WEXITSTATUS(est) == 0) {
    /* el programa terminó con éxito: tarea completada */
}
```

Siguiendo el enunciado, solo importa si el programa terminó con éxito (código 0) o no. Si el programa no existe, el hijo termina con el código 127, la convención de las shells para "comando no encontrado". El Meeseeks distingue además si el programa terminó por una señal, solo para que el mensaje sea más claro.

5. Ejecuciones

A continuación se muestran corridas de cada tipo de solicitud. Todas se hicieron con el programa final en la máquina virtual Debian, con `./box 60 10` y, para el caso A, con un tiempo de trabajo fijo de 0.2 segundos para que la salida no fuera excesivamente larga.

5.1 Caso A: solicitudes de texto

Se muestran tres corridas con dificultades distintas. Con dificultad 90 un solo Meeseeks resuelve la tarea por la regla de más de 85. Con dificultades 60 y 40 el árbol crece: se ve cómo el nivel N aumenta en cada generación, cómo el ppid de cada Meeseeks coincide con el pid de su padre y cómo, cuando uno resuelve, sus hermanos reciben la orden de terminar y la despedida sube hasta el Meeseeks original, que informa quién resolvió.

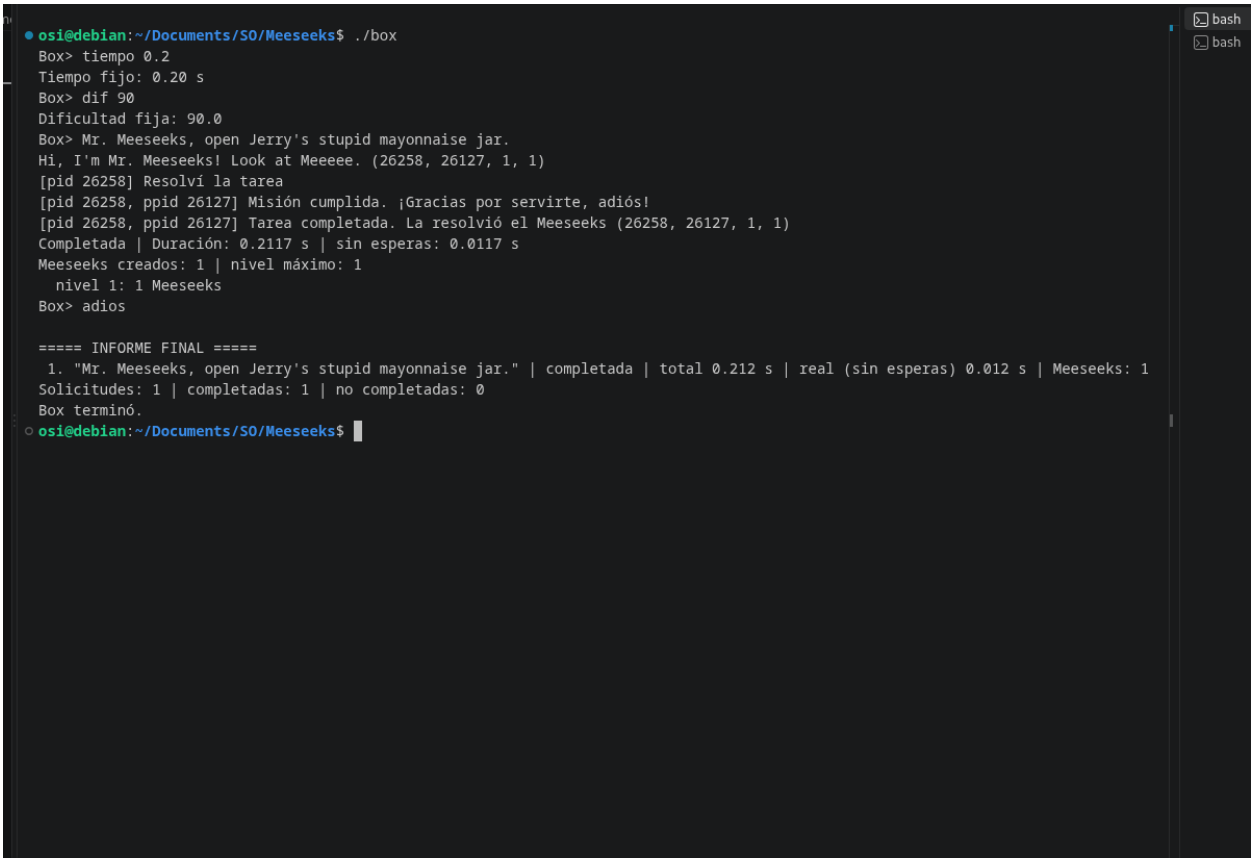


Figura 2. Caso A con dificultad 90: lo resuelve un solo Meeseeks.

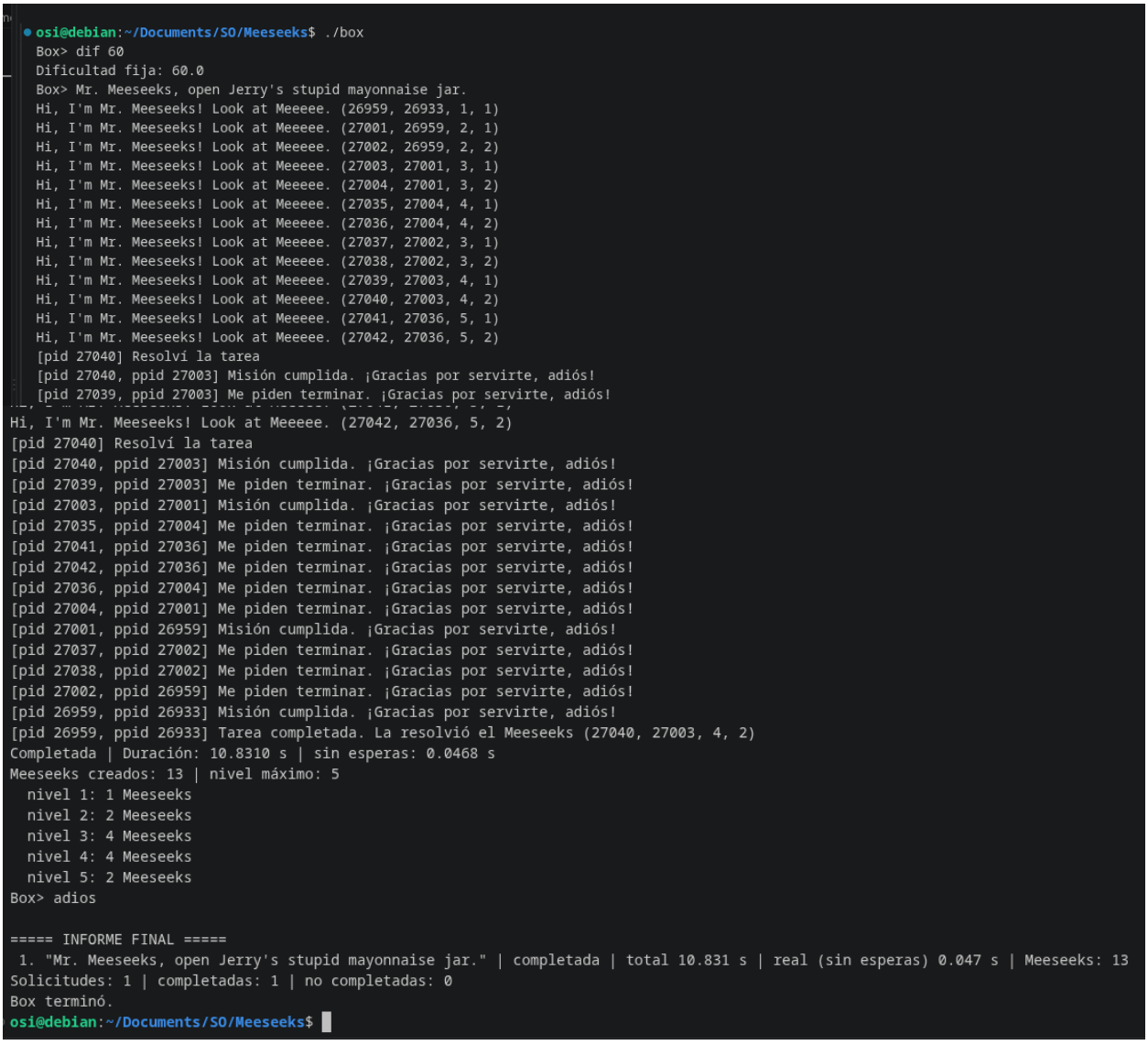


Figura 3. Caso A con dificultad 60: árbol de varios niveles y terminación en cascada.

```
osi@debian:~/Documents/SO/Meeseeks$ ./box 60 10
Box> dif 40
Dificultad fija: 40.0
Box> Mr. Meeseeks, open Jerry's stupid mayonnaise jar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27341, 27263, 1, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27353, 27341, 2, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27354, 27341, 2, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27355, 27341, 2, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27357, 27341, 2, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27356, 27341, 2, 4)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27365, 27354, 3, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27366, 27354, 3, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27368, 27354, 3, 4)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27367, 27354, 3, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27369, 27354, 3, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27391, 27365, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27392, 27365, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27393, 27356, 3, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27395, 27356, 3, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27394, 27356, 3, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27396, 27356, 3, 4)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27397, 27356, 3, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27399, 27368, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27398, 27368, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27427, 27397, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27428, 27397, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27430, 27367, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27429, 27367, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27431, 27393, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27432, 27393, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27433, 27396, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27434, 27396, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (27435, 27399, 5, 1)
    [pid 27432, ppid 27393] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27396, ppid 27356] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27393, ppid 27356] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27457, ppid 27355] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27357, ppid 27341] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27355, ppid 27341] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27353, ppid 27341] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27397, ppid 27356] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27356, ppid 27341] Me piden terminar. ¡Gracias por servirte, adiós!
    [pid 27341, ppid 27263] Misión cumplida. ¡Gracias por servirte, adiós!
    [pid 27341, ppid 27263] Tarea completada. La resolvió el Meeseeks (27460, 27429, 5, 1)
    Completada | Duración: 9.7859 s | sin esperas: 0.0584 s
    Meeseeks creados: 59 | nivel máximo: 5
      nivel 1: 1 Meeseeks
      nivel 2: 5 Meeseeks
      nivel 3: 25 Meeseeks
      nivel 4: 20 Meeseeks
      nivel 5: 8 Meeseeks
    Box> adios

    ===== INFORME FINAL =====
     1. "Mr. Meeseeks, open Jerry's stupid mayonnaise jar." | completada | total 9.786 s | real (sin esperas) 0.058 s | Meeseeks:
    Solicitudes: 1 | completadas: 1 | no completadas: 0
    Box terminó.
    osi@debian:~/Documents/SO/Meeseeks$
```

Figura 4. Caso A con dificultad 40: árbol con 5 hijos por Meeseeks y resumen por nivel.

5.2 Caso B: cálculos

Las tres corridas muestran dos cálculos correctos, en los que se respeta la precedencia de los operadores y los paréntesis, y una división entre cero, que el Meeseeks detecta y Box registra como no completada.

```
osi@debian:~/Documents/SO/Meeseeks$ ./box
Box> calc 1+2+3/2
Hi, I'm Mr. Meeseeks! Look at Meeeee. (25147, 25102, 1, 1)
[pid 25147, ppid 25102] Resultado: 4
Completada | Duración: 0.0213 s | sin esperas: 0.0213 s
Meeseeks creados: 1 | nivel máximo: 1
  nivel 1: 1 Meeseeks
Box> calc 1*2*3*4*5*6*7*8*9/0
Hi, I'm Mr. Meeseeks! Look at Meeeee. (25193, 25102, 1, 1)
[pid 25193, ppid 25102] Error: división entre cero
No completada | Duración: 0.0205 s | sin esperas: 0.0205 s
Meeseeks creados: 1 | nivel máximo: 1
  nivel 1: 1 Meeseeks
Box> calc (1*2+3*(4+5/6*7+2))
Hi, I'm Mr. Meeseeks! Look at Meeeee. (25390, 25102, 1, 1)
[pid 25390, ppid 25102] Resultado: 20
Completada | Duración: 0.0215 s | sin esperas: 0.0215 s
Meeseeks creados: 1 | nivel máximo: 1
  nivel 1: 1 Meeseeks
Box> adios

===== INFORME FINAL =====
 1. "calc 1+2+3/2" | completada | total 0.021 s | real (sin esperas) 0.021 s | Meeseeks: 1
 2. "calc 1*2*3*4*5*6*7*8*9/0" | no completada | total 0.020 s | real (sin esperas) 0.020 s | Meeseeks: 1
 3. "calc (1*2+3*(4+5/6*7+2))" | completada | total 0.021 s | real (sin esperas) 0.021 s | Meeseeks: 1
Solicitudes: 3 | completadas: 2 | no completadas: 1
Box terminó.
osi@debian:~/Documents/SO/Meeseeks$
```

Figura 5. Caso B: dos cálculos correctos y una división entre cero.

5.3 Caso C: programas externos

Las corridas incluyen un programa que termina con éxito, uno cuya duración es visible (`sleep 2`), lo que muestra que el tiempo medido incluye la ejecución real del programa externo, y uno que termina con error, que Box marca como no completado.

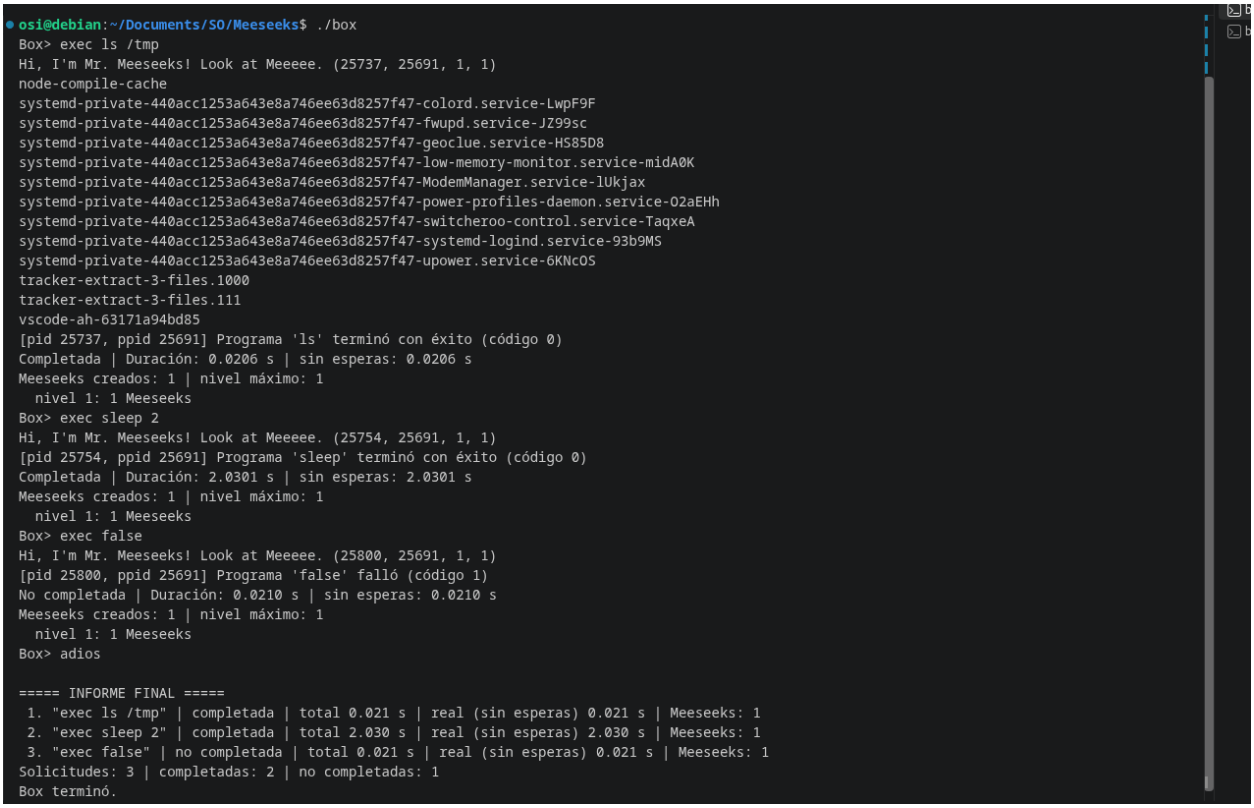


Figura 6. Caso C: programas externos con éxito y con fallo.

5.4 Tarea imposible y explosión de meeseeks

Con dificultad 0 el árbol crece hasta alcanzar el tope de procesos y luego se queda quieto, con los Meeseeks sin cupo dormidos. Al cumplirse el tiempo máximo, Box declara la explosión de meeseeks y la cascada de señales termina a todos los Meeseeks, cada uno despidiéndose después de sus hijos. La figura 8 muestra el árbol real obtenido con `pstree -p` durante esa corrida: 57 procesos, que corresponden a Box más 56 Meeseeks, la misma cifra que registró el contador de la memoria compartida.

```
osi@debian:~/Documents/SO/Meeseeks$ ./box 60 20
Box> tiempo 0.5
Tiempo fijo: 0.50 s
Box> dif 0
Dificultad fija: 0.0
Box> Mr. Meeseeks, open Jerry's stupid mayonnaise jar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6502, 6435, 1, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6503, 6502, 2, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6505, 6502, 2, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6504, 6502, 2, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6506, 6502, 2, 4)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6507, 6502, 2, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6509, 6503, 3, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6511, 6503, 3, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6508, 6503, 3, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6514, 6505, 3, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6525, 6506, 3, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6526, 6506, 3, 4)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6524, 6506, 3, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6527, 6506, 3, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6518, 6505, 3, 4)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6519, 6504, 3, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6512, 6505, 3, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6523, 6504, 3, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6520, 6506, 3, 1)
```

```
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6576, 6509, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6577, 6511, 4, 3)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6575, 6511, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6574, 6511, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6586, 6514, 4, 2)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6588, 6524, 4, 1)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6581, 6511, 4, 5)
Hi, I'm Mr. Meeseeks! Look at Meeeee. (6597, 6524, 4, 4)
explosión de meeseeks: se acabó el tiempo (20 s). Terminando todo.
[pid 6522, ppid 6505] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6519, ppid 6504] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6512, ppid 6505] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6513, ppid 6504] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6510, ppid 6505] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6518, ppid 6505] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6516, ppid 6504] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6521, ppid 6504] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6520, ppid 6506] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6528, ppid 6507] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6598, ppid 6524] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6580, ppid 6508] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6584, ppid 6508] Me piden terminar. ¡Gracias por servirte, adiós!
```

```
[pid 6508, ppid 6503] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6526, ppid 6506] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6530, ppid 6507] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6586, ppid 6514] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6527, ppid 6506] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6531, ppid 6507] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6506, ppid 6502] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6532, ppid 6507] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6533, ppid 6507] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6583, ppid 6514] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6577, ppid 6511] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6579, ppid 6511] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6581, ppid 6511] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6507, ppid 6502] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6504, ppid 6502] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6509, ppid 6503] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6511, ppid 6503] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6514, ppid 6505] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6503, ppid 6502] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6505, ppid 6502] Me piden terminar. ¡Gracias por servirte, adiós!
[pid 6502, ppid 6435] Me piden terminar. ¡Gracias por servirte, adiós!
No completada (explosión de meeseeks) | Duración: 20.0259 s | sin esperas: 20.0259 s
Meeseeks creados: 56 | nivel máximo: 4
  nivel 1: 1 Meeseeks
  nivel 2: 5 Meeseeks
  nivel 3: 25 Meeseeks
  nivel 4: 25 Meeseeks
Box> adios

===== INFORME FINAL =====
 1. "Mr. Meeseeks, open Jerry's stupid mayonnaise jar." | explosión de meeseeks | total 20.026 s | real (sin esperas) 20.026 s | Meeseeks: 56
Solicitudes: 1 | completadas: 0 | no completadas: 1
Box terminó.
```

Figura 7. Tarea de dificultad 0 terminada por explosión de meeseeks.

```
osi@debian:~/Documents/SO/Meeseeks$ pstree -p $(pgrep -o box)
box(6435)──box(6502)─┬─box(6503)─┬─box(6508)─┬─box(6580)
                     │           │           ├─box(6584)
                     │           │           ├─box(6589)
                     │           │           ├─box(6593)
                     │           │           └─box(6596)
                     │           ├─box(6509)─┬─box(6576)
                     │           │           ├─box(6578)
                     │           │           ├─box(6582)
                     │           │           ├─box(6585)
                     │           │           └─box(6587)
                     │           ├─box(6511)─┬─box(6574)
                     │           │           ├─box(6575)
                     │           │           ├─box(6577)
                     │           │           ├─box(6579)
                     │           │           └─box(6581)
                     │           ├─box(6515)
                     │           └─box(6517)
                     ├─box(6504)─┬─box(6513)
                     │           ├─box(6516)
                     │           ├─box(6519)
                     │           ├─box(6521)
                     │           └─box(6523)
                     ├─box(6505)─┬─box(6510)
                     │           ├─box(6512)
                     │           ├─box(6514)─┬─box(6583)
                     │           │           ├─box(6586)
                     │           │           ├─box(6590)
                     │           │           ├─box(6592)
                     │           │           └─box(6595)
                     │           ├─box(6518)
                     │           └─box(6522)
                     ├─box(6506)─┬─box(6520)
                     │           ├─box(6524)─┬─box(6588)
                     │           │           ├─box(6591)
                     │           │           ├─box(6594)
                     │           │           ├─box(6597)
                     │           │           └─box(6598)
                     │           ├─box(6525)
                     │           ├─box(6526)
```

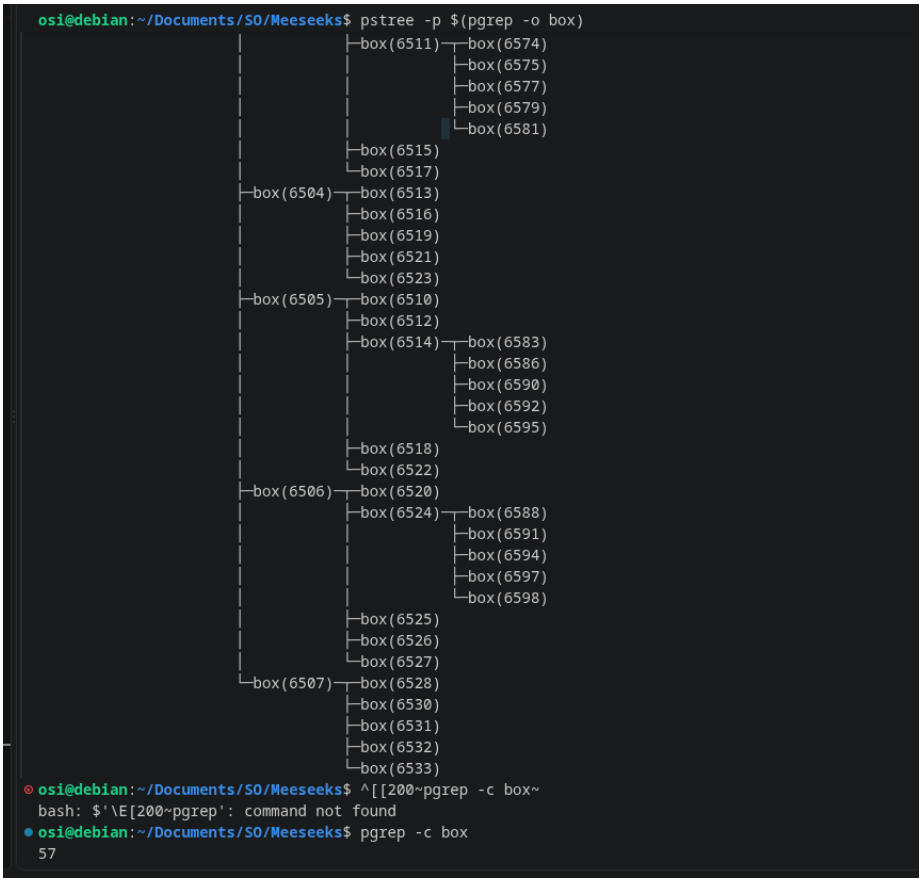


Figura 8. Árbol real de procesos durante la tarea imposible (pstree -p).

6. Experimentos y análisis (0x02)

6.1 Metodología

Todos los experimentos se ejecutaron con el programa final en una máquina virtual Debian (VirtualBox). Para no depender de escribir las solicitudes a mano, cada experimento es un script de bash (`experimentos/exp1.sh` a `exp4.sh`) que le envía a Box los comandos por la entrada estándar y guarda la salida completa en `experimentos/resultados/`. Los datos de las tablas se tomaron del informe final de Box.

Para que las comparaciones fueran válidas se controlaron las variables: la dificultad se fijó con el comando `dif` en lugar de sortearla, el tiempo de trabajo se fijó salvo en la experimento que lo estudia, y cada configuración se repitió cinco veces. Se reporta el promedio, pero también la mediana y el rango, porque, como se verá, la dispersión es alta. En los experimentos de tareas

resolubles se usó un tope de 200 procesos para que el tope no deformara el tamaño natural del árbol.

6.2 Experimento 1: dificultad contra cantidad de Meeseeks y tiempo

Pregunta: ¿cómo cambian la cantidad de Meeseeks y el tiempo al disminuir la dificultad? Configuración: tiempo fijo de 0.2 s, tope de 200 procesos, dificultades 90, 60, 40, 15 y 5, cinco repeticiones cada una.

D	Meeseeks (promedio)	Mediana	Rango	Total (s)	Sin esperas (s)
90	1.0	1	1 a 1	0.226	0.026
60	31.2	29	13 a 59	0.839	0.039
40	48.0	59	15 a 77	0.554	0.034
15	168.8	216	19 a 262	0.750	0.070
5	166.4	196	26 a 241	0.745	0.065

Tabla 2. Promedios de cinco repeticiones por dificultad.

Con dificultad 90 el comportamiento es determinista: siempre un Meeseeks, por la regla de más de 85, y la duración es prácticamente el tiempo de trabajo de 0.2 segundos. A medida que baja la dificultad, la cantidad de Meeseeks crece de forma clara, de unos 30 con dificultad 60 a cerca de 170 con dificultad 15 o 5, que es el tipo de crecimiento que describe el enunciado para las tareas difíciles.

El resultado más llamativo es la dispersión. Con dificultad 15 hubo corridas de 19 y de 262 Meeseeks. La causa es el dado de los primeros niveles: si un Meeseeks temprano acierta, la tarea termina con un árbol pequeño, y si fallan, el árbol se ramifica hasta que el aging rescata la tarea. Por eso el promedio solo no describe bien el comportamiento y conviene mirar también la mediana y el rango.

Las dificultades 15 y 5 dan resultados casi iguales. Esto ya se anticipaba con el simulador: con el dado atenuado, la tarea se resuelve sobre todo cuando el aging lleva la dificultad por encima de 85, y desde 5 o desde 15 hacen falta un número parecido de niveles (16 contra 14). Con solo cinco repeticiones, la pequeña diferencia entre ambas es ruido.

El tiempo sin esperas sube de unos 0.03 a unos 0.07 segundos cuando el árbol pasa de decenas a cientos de procesos, lo que refleja el costo de crear procesos, pipes y señales y de coordinar la cancelación. El tiempo total, en cambio, no crece en proporción a la cantidad de Meeseeks, porque trabajan en paralelo: tener 200 Meeseeks no significa sumar 200 esperas.

6.3 Experimento 2: tiempo de trabajo fijo contra aleatorio

Pregunta: ¿el tiempo de trabajo simulado cambia el comportamiento del sistema o solo la duración total? Configuración: dificultad 40, tope de 200, cinco repeticiones con tiempo fijo de 1 segundo y cinco con tiempo aleatorio entre 0.5 y 5 segundos.

Tiempo de trabajo	Meeseeks (promedio)	Mediana	Rango	Total (s)	Sin esperas (s)
Fijo, 1 s	40.8	34	6 a 93	6.28	0.033
Aleatorio, 0.5 a 5 s	34.2	27	1 a 75	4.81	0.086

Tabla 3. Promedios de cinco repeticiones con dificultad 40.

El tiempo sin esperas es pequeño en ambos casos, lo que confirma que la resta de la ruta crítica funciona: el costo real de manejar los procesos no depende de cuánto duerman los Meeseeks. El promedio del modo aleatorio es más alto (0.086 s) por una sola corrida de 0.225 s, que probablemente corresponde al tiempo de cancelar hermanos que seguían trabajando cuando llegó la éxito.

La cantidad de Meeseeks no muestra una diferencia real entre modos. En una primera ejecución de este experimento, con una versión anterior del programa, el modo aleatorio produjo más Meeseeks que el fijo, y en la ejecución final ocurrió lo contrario. Esa inversión indica que la diferencia se debe a la variabilidad del dado y no al tiempo de trabajo, lo cual es coherente con el diseño: la cantidad de Meeseeks depende de la dificultad y del dado, mientras que el tiempo solo cambia cuánto dura cada nivel.

6.4 Experimento 3: costo de cada tipo de solicitud

Pregunta: ¿cuánto cuesta atender cada tipo de solicitud? Configuración: cinco repeticiones de una misma secuencia con una solicitud de texto de dificultad 90 (tiempo fijo de 0.2 s), un cálculo, un programa exitoso, un programa que falla y una división entre cero.

Solicitud	Resultado	Meeseeks	Total (s)	Sin esperas (s)
hola (D = 90)	Completada	1	0.214	0.014
calc (4+3*2)/5	Completada	1	0.024	0.024
exec ls /tmp	Completada	1	0.022	0.022
exec false	No completada	1	0.021	0.021
calc 10/0	No completada	1	0.021	0.021

Tabla 4. Promedios de cinco repeticiones por tipo de solicitud.

Todas las solicitudes simples cuestan prácticamente lo mismo, alrededor de 21 milisegundos, sin importar si son un cálculo, un programa externo o un texto fácil, y los fallos cuestan lo mismo que los éxitos, porque el error se detecta de inmediato y se comunica con el código de salida.

Sin embargo, ese valor de 21 ms no debe interpretarse como el costo de un `fork`. Coincide con el periodo de 20 ms con que Box revisa si el Meeseeks raíz terminó (sección 3.6), y en la primera versión del programa, que esperaba con un `waitpid` bloqueante, la misma solicitud medía cerca de 1 milisegundo. La conclusión correcta es que el costo real de crear y esperar un proceso está por debajo de la resolución de la medición, y que la granularidad de 20 ms es el precio de poder vigilar el tiempo máximo de caos. Lo que de verdad encarece una solicitud no es su tipo, sino cuántos Meeseeks necesita, como mostró la experimento 1.

6.5 Experimento 4: tarea imposible y comportamiento del sistema

El enunciado plantea qué ocurre si para una solicitud muy difícil los Meeseeks no logran cumplir la misión antes del tiempo máximo y cuánto tiempo adicional se necesitaría. Para estudiarlo se lanzó una tarea de dificultad 0 con un tiempo de trabajo de 0.5 s y, mientras corría, se midió cada 5 segundos la cantidad de procesos de Box y la carga del sistema. Se probaron dos configuraciones: tope de 60 con caos a los 120 segundos y tope de 300 con caos a los 30 segundos.

Tope	Caos	Meeseeks	Procesos estables	Carga (1 min)	Duración	Limpieza
60	120 s	56	57	1.1 a 1.4	120.060 s	≈ 60 ms
300	30 s	296	297	0.3 a 0.7	30.061 s	≈ 61 ms

Tabla 5. Tarea de dificultad 0 con dos topes de procesos.

En ambos casos el árbol alcanzó el tope en menos de 10 segundos y luego se mantuvo completamente estable: la cantidad de procesos no cambió en ninguna medición posterior. La carga del sistema se mantuvo baja e incluso fue menor con 297 procesos que con 57, lo que muestra que no depende del número de Meeseeks. La explicación es que los Meeseeks sin cupo están dormidos en `pause` y no consumen procesador; la diferencia de carga entre las dos corridas proviene de otra actividad de la máquina virtual. Mantener cientos de procesos dormidos cuesta memoria y entradas en la tabla de procesos, pero casi nada de CPU.

Al cumplirse el tiempo máximo, la cascada de terminación limpió el árbol completo en unos 60 milisegundos tanto con 56 como con 296 Meeseeks, y cinco segundos después ya no quedaba ningún proceso. Que la limpieza tarde casi lo mismo con cinco veces más procesos sugiere que depende más de la profundidad del árbol que de su tamaño, porque los hermanos se despiden en paralelo; con solo dos mediciones esto queda como hipótesis.

Sobre el tiempo adicional que se necesitaría, la respuesta depende de la dificultad. Para una tarea de dificultad 0 el tiempo adicional es infinito: la dificultad nunca aumenta, el dado nunca acierta y, si no existiera el tiempo máximo, los Meeseeks esperarían para siempre o, sin tope de

procesos, seguirían multiplicándose hasta agotar los recursos de la máquina. Por eso no se dejó correr el caso sin límite. Para tareas muy difíciles pero resolubles, el aging garantiza que en una cantidad finita de niveles la dificultad supere 85; en las pruebas con dificultad 5 y tope de 200, todas las tareas se resolvieron en menos de un segundo. Una limitación del diseño es que, con un tope muy bajo, una tarea resoluble podría quedarse sin cupo antes de llegar a esos niveles y terminar igualmente por explosión de meeseeks.

7. Observaciones

La diferencia entre el simulador secuencial y el programa real fue la lección más importante del desarrollo. En el simulador los hijos se recorrían en orden y el primer éxito detenía todo, mientras que en el programa real los hermanos trabajan en paralelo y siguen creando ayudantes hasta que llega la cancelación. Por eso los mismos parámetros produjeron árboles mucho más grandes con procesos reales, y fue necesario recalibrar el aging. La comunicación entre procesos no es instantánea, y ese retraso se traduce en trabajo extra.

Varios errores aparecieron solo con árboles grandes. Sin esperar a los hijos dentro del manejador de señales quedaban procesos huérfanos; sin bloquear SIGTERM alrededor del `fork` y de la sección crítica podían darse condiciones de carrera o un bloqueo del semáforo; y un Meeseeks podía despedirse dos veces si resolvía justo cuando le llegaba la cancelación. Ninguno de estos problemas se veía con árboles de dos o tres procesos, lo que muestra que la concurrencia debe probarse con cargas grandes.

El contador de la memoria compartida coincidió en todas las corridas con la cantidad de saludos impresos y con el número de procesos que reportó el sistema operativo (`pgrep` y `pstree`), lo que respalda el uso del semáforo. El tope efectivo quedó ligeramente por debajo del configurado por la reserva "todo o nada", y el árbol resultante fue desbalanceado porque el cupo se asigna en el orden en que lo piden los Meeseeks.

Durante el desarrollo, la máquina virtual corrompió algunos objetos internos del repositorio de Git. Se recuperó eliminando los objetos vacíos y sincronizando con la copia remota en GitHub, lo cual confirmó la utilidad de hacer `push` con frecuencia.

8. Conclusiones

El programa cumple con lo solicitado: un único ejecutable en C que crea Meeseeks con `fork`, les transmite la solicitud por pipes privados, lleva los contadores en memoria compartida protegida por un semáforo, reporta pid, ppid, nivel e instancia, termina el árbol de forma ordenada hacia

atrás cuando alguien resuelve la tarea, atiende cálculos y programas externos, y protege al sistema de las tareas imposibles con un tope de procesos y un tiempo máximo configurable.

Los experimentos mostraron que la dificultad es la variable que determina el costo de una solicitud, ya que el tipo de solicitud apenas influye. También mostraron que el comportamiento del sistema es muy variable: con la misma dificultad, el tamaño del árbol puede cambiar en un orden de magnitud entre corridas, por lo que cualquier análisis de este tipo de sistemas debe basarse en varias repeticiones y en medidas de dispersión y no solo en un promedio.

El experimento de la tarea imposible dejó una conclusión práctica sobre los sistemas operativos: tener muchos procesos no es en sí un problema si están bloqueados, porque el planificador no les asigna procesador. El peligro real es la creación descontrolada de procesos, que agota memoria y entradas de la tabla de procesos, y por eso el tope de procesos es la pieza más importante del plan de contingencia.

9. Uso de Git

Todo el desarrollo se registró en el repositorio privado https://github.com/Osi200/SO-T3-Meeseeks, con commits pequeños al terminar cada etapa: el Box básico, los casos B y C, el simulador de la fórmula, el árbol con fork, los pipes de ida y de vuelta, la terminación en cascada, la memoria compartida con semáforo, el plan de contingencia, la configuración de dificultad y tiempo, el informe final y los experimentos. El programa final está en `src/meeseeksBoxFinal.c`, las versiones intermedias en `src/versiones/`, los scripts y resultados de los experimentos en `experimentos/` y las capturas en `docs/capturas/`.

Referencias

[1] N. Matthew y R. Stones, *Beginning Linux Programming*, 4.ª ed. Indianapolis, IN, EE. UU.: Wiley, 2008.

[2] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 9.ª ed. Hoboken, NJ, EE. UU.: Wiley, 2013.

TEC | Tecnológico de Costa Rica

Apéndice A: Código fuente de meeseeksBoxFinal.c

Código fuente completo del programa, tal como está en el repositorio (src/meeseeksBoxFinal.c), en fuente Courier New tamaño 8.

```
/*
 * Instituto Tecnológico de Costa Rica
 * Escuela de Ingeniería en Computación
 * IC-6600 Principios de Sistemas Operativos
 * Tarea 3 - Mr. Meeseeks
 * Estudiante: Oscar Andrés Bezara Pérez, Carné 2024075088
 * Fecha: 11 de octubre de 2026
 *
 * Descripción:
 *   Programa Box que recibe solicitudes del usuario y crea Mr. Meeseeks
 *   (procesos con fork) para atenderlas. Soporta tres tipos de solicitud:
 *     - Texto normal:   "quiero un café"  (árbol de Meeseeks, caso A)
 *     - Cálculo:        "calc 3+4*2"      (un solo Meeseeks, caso B)
 *     - Programa:       "exec ls /tmp"    (un solo Meeseeks, caso C)
 *
 * Uso:
 *   ./box [tope_procesos] [segundos_caos] [-v]
 *     tope_procesos : máximo de Meeseeks vivos a la vez (por defecto 60)
 *     segundos_caos : tiempo máximo antes del "explosión de meeseeks" (por defecto 300)
 *     -v            : muestra la dificultad D en el saludo (para depurar)
 *
 *   Comandos de configuración dentro de Box:
 *     dif aleatoria | dif preguntar | dif <0-100>
 *     tiempo aleatorio | tiempo <segundos>
 *     adios   (termina Box y muestra el informe final)
 *
 * Compilación:
 *   gcc -Wall -Wextra -pthread -o box src/meeseeksBoxFinal.c
 */

/*-------------------------------------------------------------------------
 * SECCIÓN 1: INCLUDES Y CONSTANTES
 *-------------------------------------------------------------------------*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <ctype.h>       /* isdigit, para el evaluador de operaciones */
#include <errno.h>       /* errno, para distinguir EINTR en poll */
#include <poll.h>        /* poll: vigilar varios pipes a la vez */
#include <signal.h>      /* sigaction, kill, SIGTERM */
#include <sys/mman.h>    /* mmap: memoria compartida entre procesos */
#include <semaphore.h>   /* sem_t, sem_init, sem_wait, sem_post */

#define MAX_PROCS_DEF   60    /* tope de Meeseeks vivos a la vez (configurable por argumento) */
#define CAOS_SEG_DEF    300   /* explosión de meeseeks: 5 minutos (configurable por argumento) */
#define AGING           5.0   /* cuánto sube D por nivel (la tarea se simplifica) */
#define NERF_DADO       0.1   /* factor que reduce la probabilidad del dado */
#define MAX_NIV_STATS   64    /* niveles que se contabilizan por separado */
#define MAX_HISTORIAL   256   /* solicitudes que recuerda el informe final */

/*-------------------------------------------------------------------------
```

```
 * SECCIÓN 2: TIPOS DE DATOS Y VARIABLES GLOBALES
 *--------------------------------------------------------------------------*/

/*
 * Reporte: el mensaje que viaja por el pipe de VUELTA (hijo -> padre).
 *
 * Cuando un Meeseeks resuelve la tarea, o cuando uno de sus descendientes la
 * resuelve, el reporte sube nivel por nivel hasta el Meeseeks original. Así el
 * original sabe QUIÉN resolvió (pid, ppid, N, i) sin usar variables globales,
 * como exige el enunciado.
 */
typedef struct {
    int resuelto;          /* 1 = alguien del subárbol resolvió la tarea */
    int pid, ppid, N, i;   /* datos del Meeseeks que resolvió */
    double espera_ruta;    /* segundos de simulación en la cadena raíz -> resolvedor */
} Reporte;

/*
 * Compartido: estructura que vive en MEMORIA COMPARTIDA.
 *
 * Después de un fork cada proceso tiene su propia copia de las variables, así
 * que un contador global normal no sirve: si un hijo suma 1, el padre no lo
 * ve. Esta estructura se crea con mmap(MAP_SHARED) antes de los fork, por lo
 * que todos los Meeseeks apuntan a la MISMA región física.
 *
 * Como varios procesos la modifican a la vez, TODO acceso pasa por el
 * semáforo "mutex" (exclusión mutua).
 */
typedef struct {
    sem_t mutex;                    /* semáforo: solo uno a la vez toca los contadores */
    int total;                      /* Meeseeks creados para la solicitud actual */
    int max_nivel;                  /* nivel más profundo alcanzado */
    int por_nivel[MAX_NIV_STATS];   /* cuántos Meeseeks hay en cada nivel */
    int vivos;                      /* Meeseeks vivos (o con cupo reservado) */
    int max_procs;                  /* tope configurado */
    double espera_final;            /* espera de la ruta ganadora, la escribe el raíz */
} Compartido;

/* Una entrada del historial que Box usa para el informe final */
typedef struct {
    char texto[256];
    double duracion;        /* duración total medida */
    double duracion_real;   /* duración sin las esperas simuladas */
    int completada;
    int caos;
    int meeseeks;
} Registro;

static Compartido *comp;   /* puntero a la región compartida */

/* Hijos vivos de ESTE proceso: el manejador de SIGTERM los necesita */
static pid_t mis_hijos[5];
static volatile int num_hijos = 0;

/* Banderas por proceso, protegen contra hacer algo dos veces */
static volatile sig_atomic_t ya_desregistrado = 0;   /* 1 = ya me di de baja del contador */
static volatile sig_atomic_t despedido = 0;          /* 1 = ya imprimí mi despedida */

/*
 * Configuración que Box ajusta antes de cada solicitud. Los Meeseeks la
 * heredan con el fork, así que no necesitan memoria compartida para leerla.
 */
```

```
static int    modo_dif = 0;       /* 0 aleatoria, 1 fija, 2 preguntar en cada solicitud */
static double dif_fija = 50.0;    /* valor usado cuando modo_dif == 1 */
static double dif_actual = -1;    /* dificultad de la solicitud en curso; negativo = al azar */
static int    modo_t = 0;         /* 0 tiempo aleatorio (0.5 a 5 s), 1 tiempo fijo */
static double t_fijo = 1.0;       /* segundos usados cuando modo_t == 1 */
static int    verbose = 0;        /* 1 = mostrar D en el saludo */

static Reporte meeseeks_arbol(const char *solicitud, double D, int N, int i);

/*-----------------------------------------------------------------------------
 * SECCIÓN 3: UTILIDADES GENERALES
 *-----------------------------------------------------------------------------*/

/*
 * Reloj monotónico: solo avanza hacia adelante, aunque alguien cambie la hora
 * del sistema (por ejemplo, la sincronización automática de la VM). Como la
 * tarea evalúa tiempos, necesitamos mediciones estables.
 */
static double ahora(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

/* Número aleatorio entre 0 y 1 (sin incluir el 1) */
static double rand01(void) {
    return rand() / (RAND_MAX + 1.0);
}

/* write() con el resultado "usado", para que gcc no avise por ignorarlo */
static void enviar(int fd, const void *datos, size_t n) {
    ssize_t w = write(fd, datos, n);
    (void)w;
}

/*-----------------------------------------------------------------------------
 * SECCIÓN 4: MEMORIA COMPARTIDA Y SEMÁFORO
 *-----------------------------------------------------------------------------*/

/* Se llama UNA vez en Box, antes de cualquier fork: los hijos heredan el mapeo */
static void iniciar_compartido(void) {
    /* MAP_SHARED: el fork NO copia esta región, la comparte con los hijos */
    comp = mmap(NULL, sizeof(Compartido), PROT_READ | PROT_WRITE,
                MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (comp == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }
    /* 2.° argumento = 1: el semáforo se comparte entre procesos.
       3.er argumento = 1: valor inicial 1, es decir, "libre". */
    sem_init(&comp->mutex, 1, 1);
}

/* Contadores en cero antes de cada solicitud nueva */
static void reiniciar_contadores(void) {
    comp->total = 0;
    comp->max_nivel = 0;
    comp->vivos = 1;            /* el Meeseeks raíz */
    comp->espera_final = 0;
    memset(comp->por_nivel, 0, sizeof(comp->por_nivel));
}
```

```
/*
 * Por qué se bloquea SIGTERM mientras se tiene el semáforo:
 *
 * Si llegara un SIGTERM justo cuando un proceso tiene el semáforo tomado, el
 * manejador haría _exit() sin soltarlo. Ningún otro Meeseeks podría entrar
 * nunca más a la sección crítica y todo el árbol quedaría congelado
 * (deadlock). Con la señal bloqueada, el manejador corre solo DESPUÉS de que
 * el semáforo se liberó.
 */
static void bloquear_term(sigset_t *previo) {
    sigset_t bloq;
    sigemptyset(&bloq);
    sigaddset(&bloq, SIGTERM);
    sigprocmask(SIG_BLOCK, &bloq, previo);
}

/* Cada Meeseeks se anota al nacer (los vivos ya se reservaron con reservar_hijos) */
static void registrar(int N) {
    sigset_t previo;
    bloquear_term(&previo);

    sem_wait(&comp->mutex);             /* P: entra a la sección crítica */
    comp->total++;
    if (N > comp->max_nivel) {
        comp->max_nivel = N;
    }
    if (N < MAX_NIV_STATS) {
        comp->por_nivel[N]++;
    }
    sem_post(&comp->mutex);             /* V: sale de la sección crítica */

    sigprocmask(SIG_SETMASK, &previo, NULL);
}

/* Cada Meeseeks se da de baja al desaparecer, una sola vez por proceso */
static void desregistrar(void) {
    sigset_t previo;

    if (ya_desregistrado) {
        return;
    }
    ya_desregistrado = 1;

    bloquear_term(&previo);
    sem_wait(&comp->mutex);
    comp->vivos--;
    sem_post(&comp->mutex);
    sigprocmask(SIG_SETMASK, &previo, NULL);
}

/*
 * Reserva de cupo (parte del plan de contingencia contra el caos).
 *
 * Antes de crear n hijos, el padre pide permiso: si los vivos más los nuevos
 * superarían el tope, la respuesta es NO y el Meeseeks no puede pedir ayuda.
 *
 * El cupo se reserva ANTES del fork y de forma atómica (dentro del semáforo).
 * Si se reservara después, dos padres podrían ver "queda un lugar" al mismo
 * tiempo y los dos crearían hijos, pasándose del tope.
 *
 * Consecuencia que conviene documentar: la reserva es "todo o nada" (5 o 2
 * hijos de una vez), así que el tope real queda un poco por debajo del
```

```
 * configurado, dependiendo de la forma del árbol.
 */
static int reservar_hijos(int n) {
    sigset_t previo;
    int hay_cupo;

    bloquear_term(&previo);
    sem_wait(&comp->mutex);

    if (comp->vivos + n <= comp->max_procs) {
        comp->vivos += n;
        hay_cupo = 1;
    } else {
        hay_cupo = 0;
    }

    sem_post(&comp->mutex);
    sigprocmask(SIG_SETMASK, &previo, NULL);
    return hay_cupo;
}

/*-------------------------------------------------------------------------
 * SECCIÓN 5: EVALUADOR DE OPERACIONES (CASO B)
 *
 * Método: descenso recursivo. Hay UNA función por nivel de precedencia y cada
 * una pide sus operandos a la función del nivel inmediatamente más fuerte.
 * Esa "cascada" es lo que hace que 3+4*2 dé 11 y no 14.
 *
 * De menor a mayor prioridad:
 *   expr_or  ->  expr_xor  ->  expr_and  ->  suma  ->  termino  ->  unario  ->  primario
 *      |             ^             &          + -       * / %      - ~ !      números y ( )
 *-------------------------------------------------------------------------*/

static const char *cur;        /* puntero que recorre el texto: el carácter actual */
static int hubo_error;         /* 1 si algo salió mal */
static const char *msg_error;  /* descripción de lo que salió mal */

static void error(const char *m) {
    hubo_error = 1;
    msg_error = m;
}

/* Avanza el puntero mientras haya espacios, así "3 + 4" equivale a "3+4" */
static void saltar(void) {
    while (*cur == ' ') {
        cur++;
    }
}

static long long expr_or(void);   /* declaración adelantada: los paréntesis la necesitan */

/*
 * Nivel más básico: un número o un paréntesis.
 * Un paréntesis REINICIA la precedencia: lo de adentro se evalúa desde
 * expr_or(), el nivel más bajo, como si fuera una expresión nueva.
 */
static long long primario(void) {
    saltar();

    if (*cur == '(') {
        long long v;
        cur++;
```

```
        v = expr_or();
        saltar();
        if (*cur != ')') {
            error("falta ')'");
            return 0;
        }
        cur++;
        return v;
    }

    if (isdigit((unsigned char)*cur)) {
        char *fin;
        long long v = strtoll(cur, &fin, 10);   /* lee el número y avanza cur */
        cur = fin;
        return v;
    }

    error("se esperaba un número");
    return 0;
}

/*
 * Signos pegados a un solo valor: - (negativo), ~ (NOT bit a bit), ! (NOT lógico).
 * Se llama a sí misma (recursión), por eso --5 vale 5: el segundo '-' cambia
 * el signo que había puesto el primero.
 */
static long long unario(void) {
    saltar();

    if (*cur == '-') {
        cur++;
        return -unario();
    }
    if (*cur == '~') {
        cur++;
        return ~unario();
    }
    if (*cur == '!') {
        cur++;
        return !unario();
    }
    return primario();
}

/* Multiplicación, división y módulo. Aquí se protege contra la división entre cero */
static long long termino(void) {
    long long v = unario();

    while (!hubo_error) {
        char op;
        long long d;

        saltar();
        op = *cur;
        if (op != '*' && op != '/' && op != '%') {
            break;
        }
        cur++;
        d = unario();

        if (op == '*') {
            v = v * d;
```

```
        } else if (d == 0) {
            error("división entre cero");
            return 0;
        } else if (op == '/') {
            v = v / d;
        } else {
            v = v % d;
        }
    }
    return v;
}

/* Suma y resta */
static long long suma(void) {
    long long v = termino();

    while (!hubo_error) {
        char op;
        long long d;

        saltar();
        op = *cur;
        if (op != '+' && op != '-') {
            break;
        }
        cur++;
        d = termino();

        if (op == '+') {
            v = v + d;
        } else {
            v = v - d;
        }
    }
    return v;
}

/* AND bit a bit */
static long long expr_and(void) {
    long long v = suma();

    while (!hubo_error) {
        saltar();
        if (*cur != '&') {
            break;
        }
        cur++;
        v = v & suma();
    }
    return v;
}

/* XOR bit a bit */
static long long expr_xor(void) {
    long long v = expr_and();

    while (!hubo_error) {
        saltar();
        if (*cur != '^') {
            break;
        }
        cur++;
```

```
        v = v ^ expr_and();
    }
    return v;
}

/* OR bit a bit: el nivel más bajo, es el primero que se llama y el último que opera */
static long long expr_or(void) {
    long long v = expr_xor();

    while (!hubo_error) {
        saltar();
        if (*cur != '|') {
            break;
        }
        cur++;
        v = v | expr_xor();
    }
    return v;
}

/* Devuelve 0 si salió bien (resultado en *res), 1 si hubo error */
static int evaluar(const char *texto, long long *res) {
    cur = texto;
    hubo_error = 0;
    *res = expr_or();
    saltar();

    /* Si sobró texto sin consumir, la expresión era inválida (por ejemplo "2 3") */
    if (!hubo_error && *cur != '\0') {
        error("caracteres sobrantes");
    }
    return hubo_error;
}

/*-------------------------------------------------------------------------------
 * SECCIÓN 6: DIFICULTAD, TIEMPO Y FÓRMULA DEL ÁRBOL
 *
 * ESCALA: D va de 0 a 100 y es la "probabilidad de que la tarea se pueda
 * resolver". 100 = facilísima, 0 = imposible.
 *
 * FÓRMULA PROPIA DE CADA MEESEEKS (en meeseeks_arbol):
 *   1. Si D > 85, lo resuelve él solo (regla del enunciado).
 *   2. Si no, tira un dado: gana si (número entre 0 y 100) < D * NERF_DADO.
 *      NERF_DADO = 0.1 atenúa el dado. Sin esa atenuación, las tareas
 *      difíciles se resolvían casi siempre en los primeros niveles y el
 *      árbol salía de apenas unos pocos procesos. Con el valor 0.1, la
 *      dificultad se "paga" en niveles de profundidad.
 *   3. Si falla, pide ayuda: 5 hijos si D <= 45, 2 hijos si 45 < D <= 85.
 *
 * AGING (envejecimiento): a cada hijo se le pasa D + AGING. La tarea se
 * "diluye" entre los Meeseeks y se vuelve más fácil con cada nivel, hasta que
 * D supera 85 y algún Meeseeks la resuelve con seguridad. Es la misma idea
 * del aging de planificación: lo que espera mucho gana prioridad.
 *
 * CASO IMPOSIBLE: si D == 0, el aging NO lo sube (0 se queda en 0), y el dado
 * nunca acierta (ningún número es menor que 0). Por eso la tarea no se
 * resuelve jamás, sin tener que programar un caso especial. Lo que evita el
 * colapso es el tope de procesos más el explosión de meeseeks.
 *-------------------------------------------------------------------------------*/

/* Dificultad con la que arranca el Meeseeks raíz */
static double dificultad_inicial(void) {
```

```
    if (dif_actual < 0) {
        return rand() % 101;   /* uniforme entre 0 y 100 */
    }
    return dif_actual;
}

/* Regla del enunciado: cuántos Meeseeks nuevos crea un Meeseeks que pide ayuda */
static int cantidad_hijos(double D) {
    if (D <= 45.0) {
        return 5;   /* tarea muy difícil: pide 5 ayudantes */
    }
    return 2;       /* tarea media (45 a 85): pide 2 ayudantes */
}

/* Aging: dificultad que reciben los hijos */
static double dificultad_hijo(double D) {
    double D_hijo;

    if (D == 0) {
        D_hijo = 0;            /* imposible: nunca se simplifica */
    } else {
        D_hijo = D + AGING;    /* la tarea se simplifica un poco por nivel */
    }

    if (D_hijo > 100.0) {
        D_hijo = 100.0;        /* 100 es el máximo de la escala */
    }
    return D_hijo;
}

/* Simula el tiempo de trabajo y devuelve cuánto durmió (para restarlo después) */
static double esperar_trabajo(void) {
    double segundos;

    if (modo_t == 0) {
        segundos = 0.5 + rand01() * 4.5;   /* aleatorio entre 0.5 y 5 s */
    } else {
        segundos = t_fijo;                 /* valor elegido por el usuario */
    }

    usleep((useconds_t)(segundos * 1e6));
    return segundos;
}

/*------------------------------------------------------------------------------
 * SECCIÓN 7: TERMINACIÓN EN CASCADA (SEÑALES)
 *
 * Por qué SIGTERM y no otro mecanismo:
 *   Cuando un Meeseeks resuelve la tarea, sus hermanos (que siguen trabajando
 *   o durmiendo) deben enterarse y despedirse. Hay dos opciones: avisarles
 *   por pipe o con una señal. Un pipe obligaría a cada Meeseeks a estar
 *   leyendo mientras trabaja. Una señal, en cambio, interrumpe al proceso
 *   aunque esté dormido en usleep() o en pause().
 *
 * Cómo funciona la cascada:
 *   1. El padre recibe por pipe el primer reporte de éxito.
 *   2. Manda SIGTERM a los hermanos que siguen vivos.
 *   3. Cada hermano, en su manejador (al_terminar), manda SIGTERM a SUS
 *      hijos y ESPERA con wait() a que se despidan antes de despedirse él.
 *   4. Así la despedida va de abajo hacia arriba y nadie queda huérfano
 *      (sin ese wait, un padre moría antes que sus hijos y estos eran
 *      adoptados por init).
```

```
 *
 * El mismo mecanismo se usa para el explosión de meeseeks: Box manda SIGTERM al
 * Meeseeks raíz y la cascada baja por todo el árbol.
 *
 * Se usa write() y _exit() en vez de printf() y exit() porque solo algunas
 * funciones son seguras dentro de un manejador de señales; _exit tampoco
 * vacía buffers heredados del padre, lo que evitaría texto duplicado.
 *-----------------------------------------------------------------------------*/

static void al_terminar(int sig) {
    (void)sig;

    /* Propaga la señal hacia abajo */
    for (int k = 0; k < num_hijos; k++) {
        kill(mis_hijos[k], SIGTERM);
    }

    /* Espera a que los hijos se despidan primero */
    while (wait(NULL) > 0) {
        /* nada: solo recogemos hijos */
    }

    /* Si ya me despedí como resolvedor, no repito la despedida */
    if (!despedido) {
        char msg[128];
        int n;

        despedido = 1;
        n = snprintf(msg, sizeof(msg),
                     "[pid %d, ppid %d] Me piden terminar. ¡Gracias por servirte, adiós!\n",
                     getpid(), getppid());
        enviar(STDOUT_FILENO, msg, n);
    }

    desregistrar();
    _exit(EXIT_FAILURE);
}

/* Instala al_terminar como respuesta a SIGTERM */
static void instalar_manejador(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = al_terminar;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGTERM, &sa, NULL);
}

/*-----------------------------------------------------------------------------
 * SECCIÓN 8: ÁRBOL DE MEESEEKS (CASO A) CON PIPES
 *
 * Comunicación entre procesos (sin variables globales para mensajes):
 *
 *   PIPE DE IDA (padre -> hijo): el padre le pasa la solicitud al hijo. Se
 *   crea ANTES del fork para que el hijo herede los dos extremos. El padre
 *   escribe y cierra; el hijo lee. Es un mensaje privado por cada hijo.
 *
 *   PIPE DE VUELTA (hijo -> padre): el hijo devuelve su Reporte. Un pipe es
 *   unidireccional, por eso hace falta uno distinto para cada dirección.
 *
 * Regla de oro de los pipes: cada proceso cierra los extremos que no usa. Un
 * read() solo ve el fin de archivo cuando TODOS los extremos de escritura
 * están cerrados; si un hijo dejara abierto un extremo ajeno, el padre
```

```
 * esperaría para siempre.
 *
 * El padre vigila los pipes de vuelta de todos sus hijos a la vez con poll(),
 * para reaccionar al PRIMERO que reporte éxito (y no al primero de la lista).
 *--------------------------------------------------------------------------*/

/*
 * Crea los hijos (todos juntos, en paralelo), les manda la solicitud por el
 * pipe de ida y recoge sus reportes por el pipe de vuelta.
 * Al primer éxito avisa a los hermanos con SIGTERM. Devuelve el reporte del
 * subárbol.
 */
static Reporte crear_hijos(const char *solicitud, double D, int N) {
    int hijos = cantidad_hijos(D);
    double D_hijo = dificultad_hijo(D);
    int ida[5][2];      /* ida:    padre -> hijo */
    int vuelta[5][2];   /* vuelta: hijo -> padre */
    int creados = 0;
    sigset_t bloq, previo;

    sigemptyset(&bloq);
    sigaddset(&bloq, SIGTERM);
    num_hijos = 0;

    /*---------------- Creación de los hijos ----------------*/
    for (int k = 0; k < hijos; k++) {
        pid_t p;

        /* Los pipes se crean ANTES del fork para que el hijo los herede */
        if (pipe(ida[k]) < 0 || pipe(vuelta[k]) < 0) {
            perror("pipe");
            break;
        }

        /*
         * SIGTERM bloqueado alrededor del fork: así ninguna señal llega a un
         * hijo recién nacido que todavía no instaló su manejador, ni a un
         * padre que todavía no anotó al hijo en su lista mis_hijos.
         */
        sigprocmask(SIG_BLOCK, &bloq, &previo);
        p = fork();
        if (p < 0) {
            perror("fork");
            sigprocmask(SIG_SETMASK, &previo, NULL);
            break;
        }

        if (p == 0) {
            /*------------ HIJO ------------*/
            char buf[256];
            ssize_t n;
            Reporte r;

            num_hijos = 0;   /* la lista heredada es de mi padre, no mía */
            instalar_manejador();
            sigprocmask(SIG_SETMASK, &previo, NULL);   /* ahora sí acepto señales */

            close(ida[k][1]);      /* no escribo en la ida */
            close(vuelta[k][0]);   /* no leo de la vuelta */
            for (int j = 0; j < k; j++) {
                close(vuelta[j][0]);   /* lecturas de hermanos anteriores que heredé */
            }
```

```
        /* Recibo la solicitud por el pipe (y no por la memoria copiada) */
        n = read(ida[k][0], buf, sizeof(buf) - 1);
        close(ida[k][0]);
        if (n < 0) {
            n = 0;
        }
        buf[n] = '\0';

        srand(getpid());   /* si no, todos los hermanos tirarían el mismo dado */

        r = meeseeks_arbol(buf, D_hijo, N + 1, k + 1);
        enviar(vuelta[k][1], &r, sizeof(r));   /* reporto al padre */
        close(vuelta[k][1]);
        desregistrar();

        /* El hijo NUNCA debe volver al código del padre: termina aquí */
        if (r.resuelto) {
            exit(EXIT_SUCCESS);
        }
        exit(EXIT_FAILURE);
    }

    /*------------ PADRE ------------*/
    mis_hijos[num_hijos++] = p;   /* anoto al hijo ANTES de aceptar señales */
    sigprocmask(SIG_SETMASK, &previo, NULL);
    creados++;

    close(ida[k][0]);                                   /* no leo de la ida */
    enviar(ida[k][1], solicitud, strlen(solicitud));   /* le mando la solicitud */
    close(ida[k][1]);                                   /* cerrar avisa "fin del mensaje" */
    close(vuelta[k][1]);                                /* solo leo de la vuelta */
}

/*---------------- Espera de reportes con poll ----------------*/
struct pollfd fds[5];
int vivo[5];   /* 1 = hijo que todavía no ha reportado */
Reporte final = {0, 0, 0, 0, 0, 0.0};
int abiertos = creados;

for (int k = 0; k < creados; k++) {
    fds[k].fd = vuelta[k][0];
    fds[k].events = POLLIN;
    vivo[k] = 1;
}

while (abiertos > 0) {
    /* Duerme hasta que algún pipe tenga algo que leer */
    if (poll(fds, creados, -1) < 0) {
        if (errno == EINTR) {
            continue;   /* una señal interrumpió poll: reintenta */
        }
        break;
    }

    for (int k = 0; k < creados; k++) {
        Reporte r;
        ssize_t n;

        if (fds[k].fd < 0) {
            continue;   /* poll ignora descriptores negativos: ya cerrado */
        }
```

```
            if (!(fds[k].revents & (POLLIN | POLLHUP))) {
                continue;   /* este pipe no tiene novedades */
            }

            n = read(fds[k].fd, &r, sizeof(r));
            if (n == (ssize_t)sizeof(r)) {
                /* Llegó un reporte completo */
                vivo[k] = 0;
                if (r.resuelto && !final.resuelto) {
                    /* Primer éxito: lo guardo y aviso a los hermanos que sigan vivos */
                    final = r;
                    for (int j = 0; j < creados; j++) {
                        if (vivo[j]) {
                            kill(mis_hijos[j], SIGTERM);
                        }
                    }
                }
            } else {
                /* Fin de archivo: el hijo cerró su extremo, no habrá más datos */
                close(fds[k].fd);
                fds[k].fd = -1;
                vivo[k] = 0;
                abiertos--;
            }
        }
    }

    /* Recoge a todos los hijos: no quedan zombies */
    while (wait(NULL) > 0) {
        /* nada: solo recogemos hijos */
    }
    num_hijos = 0;
    return final;
}

/*
 * Un Meeseeks del caso A: saluda, trabaja, tira el dado y, si falla, pide
 * ayuda. Devuelve el reporte de su subárbol.
 *
 * Tiempo "real": cada Meeseeks que duerme suma su espera al reporte mientras
 * este sube por la cadena raíz -> resolvedor (ruta crítica). Como los
 * Meeseeks trabajan en paralelo, sus esperas se solapan y no se pueden
 * restar todas; solo la espera de la cadena ganadora aparece de verdad en el
 * tiempo total.
 */
static Reporte meeseeks_arbol(const char *solicitud, double D, int N, int i) {
    double mi_espera;
    int hijos_necesarios;
    Reporte r;

    registrar(N);

    if (verbose) {
        printf("Hi, I'm Mr. Meeseeks! Look at Meeeee. (%d, %d, %d, %d) D=%.1f\n",
               getpid(), getppid(), N, i, D);
    } else {
        printf("Hi, I'm Mr. Meeseeks! Look at Meeeee. (%d, %d, %d, %d)\n",
               getpid(), getppid(), N, i);
    }
    fflush(stdout);   /* vaciar el buffer antes de cualquier fork */

    mi_espera = esperar_trabajo();   /* simula el tiempo de trabajo */
```

```
    hijos_necesarios = cantidad_hijos(D);

    /* ¿Lo resuelvo yo solo? Regla fija (D > 85) o dado atenuado */
    if (D > 85.0 || rand01() * 100.0 < D * NERF_DADO) {
        despedido = 1;
        printf("[pid %d] Resolví la tarea\n", getpid());
        printf("[pid %d, ppid %d] Misión cumplida. ¡Gracias por servirte, adiós!\n",
               getpid(), getppid());
        fflush(stdout);

        r.resuelto = 1;
        r.pid = getpid();
        r.ppid = getppid();
        r.N = N;
        r.i = i;
        r.espera_ruta = mi_espera;
        return r;
    }

    /* No pude solo: pido cupo para crear ayudantes */
    if (!reservar_hijos(hijos_necesarios)) {
        /*
         * Sin cupo: no puedo pedir ayuda. Duermo sin gastar CPU hasta que
         * llegue SIGTERM (de mi padre, porque alguien resolvió, o del caos
         * planetario). Así los Meeseeks sobrantes de una tarea imposible
         * quedan quietos y no consumen procesador.
         */
        pause();
        r.resuelto = 0;
        r.pid = 0;
        r.ppid = 0;
        r.N = 0;
        r.i = 0;
        r.espera_ruta = 0.0;
        return r;
    }

    r = crear_hijos(solicitud, D, N);
    if (r.resuelto) {
        /* Un descendiente resolvió: yo también me despido */
        r.espera_ruta += mi_espera;   /* sumo mi espera a la ruta ganadora */
        despedido = 1;
        printf("[pid %d, ppid %d] Misión cumplida. ¡Gracias por servirte, adiós!\n",
               getpid(), getppid());
        fflush(stdout);
    }
    return r;
}

/*-------------------------------------------------------------------------
 * SECCIÓN 9: LOS TRES TIPOS DE SOLICITUD
 *-------------------------------------------------------------------------*/

/* Saludo de los Meeseeks de los casos B y C (el caso A saluda en meeseeks_arbol) */
static void saludar(int N, int i) {
    registrar(N);
    printf("Hi, I'm Mr. Meeseeks! Look at Meeeee. (%d, %d, %d, %d)\n",
           getpid(), getppid(), N, i);
    fflush(stdout);
}

/*
```

```
 * Caso B: solicitud de cálculo. La resuelve un solo Meeseeks.
 * El código de salida le dice a Box si funcionó: 0 = completada, 1 = error.
 */
static void caso_calculo(const char *expresion, int N, int i) {
    long long res;

    saludar(N, i);
    if (evaluar(expresion, &res) == 0) {
        printf("[pid %d, ppid %d] Resultado: %lld\n", getpid(), getppid(), res);
        exit(EXIT_SUCCESS);
    }
    printf("[pid %d, ppid %d] Error: %s\n", getpid(), getppid(), msg_error);
    exit(EXIT_FAILURE);
}

/*
 * Caso C: programa externo, por ejemplo "exec ls /tmp".
 *
 * exec REEMPLAZA el proceso entero por el programa nuevo. Si el Meeseeks
 * hiciera exec directamente, desaparecería sin poder reportar nada. Por eso
 * hace su propio fork: el hijo ejecuta el programa y el Meeseeks espera con
 * waitpid para leer el valor de retorno (0 = éxito, otro valor = fallo).
 * Quedan tres niveles de procesos: Box, el Meeseeks y el programa externo.
 */
static void caso_programa(const char *comando, int N, int i) {
    char copia[256];   /* copia porque strtok modifica el texto */
    char *args[32];
    char *tok;
    int n = 0;
    int est;
    pid_t pid_ext;

    saludar(N, i);

    strncpy(copia, comando, sizeof(copia) - 1);
    copia[sizeof(copia) - 1] = '\0';

    /* Parte "ls /tmp" en {"ls", "/tmp", NULL}, el formato que pide execvp */
    tok = strtok(copia, " ");
    while (tok != NULL && n < 31) {
        args[n++] = tok;
        tok = strtok(NULL, " ");
    }
    args[n] = NULL;   /* execvp exige que el arreglo termine en NULL */

    if (n == 0) {
        printf("[pid %d, ppid %d] Error: no indicaste ningún programa\n",
               getpid(), getppid());
        exit(EXIT_FAILURE);
    }

    fflush(stdout);
    pid_ext = fork();
    if (pid_ext < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid_ext == 0) {
        execvp(args[0], args);   /* si funciona, esta línea nunca retorna */
        /* Si llegamos aquí, exec falló (el programa no existe, sin permisos...) */
        fprintf(stderr, "[pid %d] No se pudo ejecutar '%s'\n", getpid(), args[0]);
```

```
        _exit(127);             /* 127 = "comando no encontrado" por convención */
    }

    waitpid(pid_ext, &est, 0);

    if (WIFEXITED(est) && WEXITSTATUS(est) == 0) {
        printf("[pid %d, ppid %d] Programa '%s' terminó con éxito (código 0)\n",
               getpid(), getppid(), args[0]);
        exit(EXIT_SUCCESS);
    }

    if (WIFEXITED(est)) {
        printf("[pid %d, ppid %d] Programa '%s' falló (código %d)\n",
               getpid(), getppid(), args[0], WEXITSTATUS(est));
    } else {
        printf("[pid %d, ppid %d] Programa '%s' murió por una señal\n",
               getpid(), getppid(), args[0]);
    }
    exit(EXIT_FAILURE);
}

/*
 * Caso A: solicitud de texto. Este es el Meeseeks RAÍZ del árbol.
 * Toma la dificultad inicial, lanza el árbol y, al final, informa quién
 * resolvió la tarea.
 */
static void caso_texto(const char *solicitud, int N, int i) {
    double D;
    Reporte r;

    srand(getpid() ^ time(NULL));
    D = dificultad_inicial();

    instalar_manejador();   /* el raíz también debe poder despedirse y propagar */
    r = meeseeks_arbol(solicitud, D, N, i);

    if (r.resuelto) {
        comp->espera_final = r.espera_ruta;   /* Box la lee para restarla del tiempo total */
        printf("[pid %d, ppid %d] Tarea completada. La resolvió el Meeseeks (%d, %d, %d, %d)\n",
               getpid(), getppid(), r.pid, r.ppid, r.N, r.i);
        desregistrar();
        exit(EXIT_SUCCESS);
    }

    printf("[pid %d, ppid %d] No se pudo resolver la tarea\n", getpid(), getppid());
    desregistrar();
    exit(EXIT_FAILURE);
}

/*
 * Un Meeseeks raíz: decide qué tipo de solicitud es según su prefijo y la
 * atiende. Nunca retorna: cada caso termina con exit().
 *
 * Se usan prefijos ("calc ", "exec ") y no se intenta adivinar el tipo,
 * porque adivinar es frágil: "2 cafés" tiene un número y no es un cálculo.
 */
static void meeseeks(const char *solicitud, int N, int i) {
    if (strncmp(solicitud, "calc ", 5) == 0) {
        caso_calculo(solicitud + 5, N, i);
    }
    if (strncmp(solicitud, "exec ", 5) == 0) {
        caso_programa(solicitud + 5, N, i);
```

```
    }
    caso_texto(solicitud, N, i);
}

/*-------------------------------------------------------------------------------
 * SECCIÓN 10: FUNCIONES DE APOYO DE BOX
 *-------------------------------------------------------------------------------*/

/*
 * Procesa los comandos de configuración ("dif ..." y "tiempo ...").
 * Devuelve 1 si la línea era un comando (y ya se atendió), 0 si no.
 */
static int procesar_configuracion(const char *linea) {
    if (strncmp(linea, "dif ", 4) == 0) {
        const char *arg = linea + 4;

        if (strcmp(arg, "aleatoria") == 0) {
            modo_dif = 0;
            printf("Dificultad: aleatoria\n");
        } else if (strcmp(arg, "preguntar") == 0) {
            modo_dif = 2;
            printf("Dificultad: se preguntará en cada solicitud\n");
        } else {
            char *fin;
            double v = strtod(arg, &fin);
            if (fin == arg || v < 0 || v > 100) {
                printf("Valor inválido: usa aleatoria, preguntar o un número de 0 a 100\n");
            } else {
                modo_dif = 1;
                dif_fija = v;
                printf("Dificultad fija: %.1f\n", v);
            }
        }
        return 1;
    }

    if (strncmp(linea, "tiempo ", 7) == 0) {
        const char *arg = linea + 7;

        if (strcmp(arg, "aleatorio") == 0) {
            modo_t = 0;
            printf("Tiempo: aleatorio de 0.5 a 5 s\n");
        } else {
            char *fin;
            double v = strtod(arg, &fin);
            if (fin == arg || v < 0) {
                printf("Valor inválido: usa aleatorio o segundos (ej. 0.2)\n");
            } else {
                modo_t = 1;
                t_fijo = v;
                printf("Tiempo fijo: %.2f s\n", v);
            }
        }
        return 1;
    }

    return 0;
}

/*
 * Decide la dificultad de la solicitud que se va a lanzar (el Meeseeks raíz la
 * hereda con el fork). Devuelve 0 si todo bien, 1 si se acabó la entrada.
```

```
 */
static int decidir_dificultad(void) {
    char resp[64];
    double v;

    if (modo_dif == 0) {
        dif_actual = -1;           /* al azar: la tira el Meeseeks raíz */
        return 0;
    }
    if (modo_dif == 1) {
        dif_actual = dif_fija;     /* valor fijo elegido por el usuario */
        return 0;
    }

    /* modo_dif == 2: se pregunta en cada solicitud */
    printf("Dificultad de esta solicitud (0 a 100): ");
    fflush(stdout);
    if (fgets(resp, sizeof(resp), stdin) == NULL) {
        return 1;
    }

    v = atof(resp);
    if (v < 0) {
        v = 0;
    }
    if (v > 100) {
        v = 100;
    }
    dif_actual = v;
    return 0;
}

/*
 * Espera al Meeseeks raíz con un límite de tiempo. Si se cumple, se declara el
 * explosión de meeseeks: se manda SIGTERM al raíz y la cascada de señales termina
 * todo el árbol (sección 7).
 *
 * Se usa waitpid con WNOHANG (no bloquea) y se revisa cada 20 ms: así Box
 * puede vigilar el reloj sin gastar CPU.
 */
static void esperar_con_limite(pid_t pid, double t0, int caos_seg, int *ok, int *caos) {
    int estado = 0;
    double limite = t0 + caos_seg;

    *ok = 0;
    *caos = 0;

    while (1) {
        pid_t r = waitpid(pid, &estado, WNOHANG);

        if (r == pid) {
            /* El Meeseeks raíz terminó: éxito si salió normal con código 0 */
            if (WIFEXITED(estado) && WEXITSTATUS(estado) == 0) {
                *ok = 1;
            }
            return;
        }

        if (ahora() > limite) {
            printf("explosión de meeseeks: se acabó el tiempo (%d s). Terminando todo.\n",
caos_seg);
            fflush(stdout);
```

```
            kill(pid, SIGTERM);        /* la cascada baja por todo el árbol */
            waitpid(pid, &estado, 0);  /* espera a que el raíz termine de despedirse */
            *caos = 1;
            return;
        }

        usleep(20000);   /* revisa cada 20 ms sin gastar CPU */
    }
}

/*-----------------------------------------------------------------------------
 * SECCIÓN 11: PROGRAMA PRINCIPAL (BOX)
 *-----------------------------------------------------------------------------*/

int main(int argc, char *argv[]) {
    int max_procs = MAX_PROCS_DEF;
    int caos_seg = CAOS_SEG_DEF;
    char linea[256];             /* lo que escribe el usuario */
    Registro hist[MAX_HISTORIAL];
    int n_hist = 0;

    /*---------------- Argumentos: ./box [tope] [segundos_caos] [-v] ----------------*/
    if (argc > 1) {
        max_procs = atoi(argv[1]);
    }
    if (argc > 2) {
        caos_seg = atoi(argv[2]);
    }
    if (argc > 3 && strcmp(argv[3], "-v") == 0) {
        verbose = 1;
    }
    if (max_procs < 1) {
        max_procs = MAX_PROCS_DEF;   /* valor inválido: usa el predeterminado */
    }
    if (caos_seg < 1) {
        caos_seg = CAOS_SEG_DEF;
    }

    iniciar_compartido();   /* antes de cualquier fork: los hijos heredan el mapeo */
    comp->max_procs = max_procs;

    /*---------------- Ciclo principal: una solicitud tras otra ----------------*/
    while (1) {
        double t0, dur, real;
        int ok, caos;
        pid_t pid;

        printf("Box> ");
        fflush(stdout);

        if (fgets(linea, sizeof(linea), stdin) == NULL) {
            break;   /* fin de la entrada (Ctrl+D) */
        }
        linea[strcspn(linea, "\n")] = '\0';   /* quita el salto de línea */

        if (strcmp(linea, "adios") == 0) {
            break;   /* el usuario se despide */
        }
        if (linea[0] == '\0') {
            continue;   /* solo presionó Enter: no se crea ningún Meeseeks */
        }
        if (procesar_configuracion(linea)) {
```

```
            continue;   /* era un comando de configuración */
        }
        if (decidir_dificultad() != 0) {
            break;
        }

        reiniciar_contadores();

        t0 = ahora();
        fflush(stdout);   /* evita que el hijo herede texto pendiente y lo duplique */
        pid = fork();     /* aquí nace el primer Meeseeks */

        if (pid < 0) {
            perror("fork");
            continue;
        }
        if (pid == 0) {
            meeseeks(linea, 1, 1);   /* hijo: nivel N=1, instancia i=1 */
        }

        /* Box espera al Meeseeks raíz, con límite de tiempo */
        esperar_con_limite(pid, t0, caos_seg, &ok, &caos);

        dur = ahora() - t0;
        real = dur - comp->espera_final;   /* tiempo sin las esperas simuladas */
        if (real < 0) {
            real = 0;
        }

        /* Guarda la solicitud en el historial para el informe final */
        if (n_hist < MAX_HISTORIAL) {
            Registro *g = &hist[n_hist];
            n_hist++;
            snprintf(g->texto, sizeof(g->texto), "%s", linea);
            g->duracion = dur;
            g->duracion_real = real;
            g->completada = (ok && !caos);
            g->caos = caos;
            g->meeseeks = comp->total;
        }

        /* Resumen de esta solicitud */
        if (caos) {
            printf("No completada (explosión de meeseeks)");
        } else if (ok) {
            printf("Completada");
        } else {
            printf("No completada");
        }
        printf(" | Duración: %.4f s | sin esperas: %.4f s\n", dur, real);

        printf("Meeseeks creados: %d | nivel máximo: %d\n", comp->total, comp->max_nivel);
        for (int n = 1; n <= comp->max_nivel && n < MAX_NIV_STATS; n++) {
            printf("  nivel %d: %d Meeseeks\n", n, comp->por_nivel[n]);
        }
    }

    /*---------------- Informe final ----------------*/
    printf("\n===== INFORME FINAL =====\n");
    int completadas = 0;

    for (int k = 0; k < n_hist; k++) {
```

```
        Registro *g = &hist[k];
        const char *estado;

        if (g->completada) {
            completadas++;
            estado = "completada";
        } else if (g->caos) {
            estado = "explosión de meeseeks";
        } else {
            estado = "no completada";
        }

        printf("%2d. \"%s\" | %s | total %.3f s | real (sin esperas) %.3f s | Meeseeks: %d\n",
               k + 1, g->texto, estado, g->duracion, g->duracion_real, g->meeseeks);
    }
    printf("Solicitudes: %d | completadas: %d | no completadas: %d\n",
           n_hist, completadas, n_hist - completadas);

    /*---------------- Limpieza ----------------*/
    printf("Box terminó.\n");
    sem_destroy(&comp->mutex);
    munmap(comp, sizeof(Compartido));
    return 0;
}
```